DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LANGUAGE TRANSLATORS

V SEMESTER

SYLLABUS

Course Objectives:

1. To understand the machine dependent and machine independent features of assemblers/loaders.
2. To gain knowledge on data structures required for implementation of system software like assemblers/loaders/compliers.
3. To understand the role of loaders and linkers in program execution and linking.
4. To understand the various phases of designing a compiler.
5. To use formal attributed grammars for specifying the syntax and semantics of programming languages, and their impact on compiler design.
6. To gain knowledge of how to design an assembler/compliers.

Course Outcomes:

On successful completion of the module students will be able to:

1. An ability to design and implement assemblers for different computer architectures.
2. An ability to design and implement loaders.
3. An ability to understand the major phases of compilation, particularly lexical analysis, parsing, semantic analysis, and code generation.
4. The ability to use formal attributed grammars for specifying the syntax and semantics of programming languages, and their impact on compiler design.
5. An ability to understand how the machine code translation occurs.
6. An ability to develop system programs.

UNIT – I

Introduction to System Software and Machine Structure: System programs – Assembler, Interpreter, Operating system. Machine Structure – instruction set and addressing modes. **Assemblers:** Basic assembler functions, machine – dependent and machine independent assembler features. Assembler design – Two-pass assembler with overlay structure, one – pass assembler and multi-pass assembler.

UNIT – II

Loaders and Linkers: Basic loader functions, machine – dependent and machine – independent Loader features. Loader design – Linkage editors, dynamic linking and bootstrap loaders.

UNIT – III

Source Program Analysis: Compilers – Analysis of the Source Program – Phases of a Compiler – Cousins of Compiler – Grouping of Phases – Compiler Construction Tools.

Lexical Analysis: Role of Lexical Analyzer – Input Buffering – Specification of Tokens – Recognition of Tokens –A Language for Specifying Lexical Analyzer.

UNIT – IV

Parsing: Role of Parser – Context free Grammars – Writing a Grammar – Predictive Parser – LR Parser.

Intermediate Code Generation: Intermediate Languages – Declarations – Assignment Statements – Boolean Expressions – Case Statements – Back Patching – Procedure Calls.

UNIT – V

Basic Optimization: Constant-Expression Evaluation – Algebraic Simplifications and Re-association– Copy Propagation – Common Sub-expression Elimination – Loop-Invariant Code Motion – Induction Variable Optimization.

Code Generation: Issues in the Design of Code Generator – The Target Machine – Runtime Storage management – Next-use Information – A simple Code Generator – DAG Representation of Basic Blocks – Peephole Optimization – Generating Code from DAGs.

TEXT BOOKS

1. Alfred Aho, V. Ravi Sethi, and D. Jeffery Ullman, "Compilers Principles, Techniques and Tools", Addison-Wesley, 1988.
2. Leland L. Beck, "System Software – In Introduction to System Programming", Addison-Wesley, 1990.

REFERENCE BOOKS

1. Allen Holub, "Compiler Design in C", Prentice-Hall of India, 1990.
2. Charles N. Fischer and Richard J. Leblanc, "Crafting a Compiler with C", Benjamin Cummings, 1991.
3. Steven S. Muchnick, "Advanced Compiler Design Implementation", Morgan Koffman, 1997.
4. Damdhare, "Introduction to System Software", McGraw Hill, 1986.

WEBSITE

1. http://www.compilerconnection.com/
2. news:comp.compilers
3. http://ise2008.blogspot.in/2010/09/system-software-leland-beck-ppts.html

SRI MANAKULA VINAYAGAR ENGINEERING COLLEGE
(Approved by AICTE, New Delhi & Affiliated to Pondicherry University)
(Accredited by NBA-AICTE, New Delhi, ISO 9001:2000 Certified Institution & Accredited by NAAC with "A" Grade)
Madagadipet, Puducherry - 605 107

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SUBJECT NAME: LANGUAGE TRANSLATORS **SUBJECT CODE: CST54**

Prepared By:

Mr. M. SHANMUGAM, AP/CSE

Mr. A. PRAKASH, AP/CSE

Verified by: **Approved by:**

UNIT - I

Introduction to System Software and Machine Structure: System programs – Assembler, Interpreter, Operating system. Machine Structure – instruction set and addressing modes. **Assemblers:** Basic assembler functions, machine – dependent and machine independent assembler features. Assembler design – Two-pass assembler with overlay structure, one – pass assembler and multi-pass assembler.

2 MARKS

1. What is language translator? **(NOV 2014)**

A language translator is a program that takes as input a program written in one language and produces as output a program in another language. In program translation, the translator performs another very important role, the error-detection. Any violation of the High Level Language (HLL) specification would be detected and reported to the programmers.

2. What are the roles of translators?

Important role of translator are:

1. Translating the HLL program input into an equivalent Machine Language (ML) program.
2. Providing diagnostic messages wherever the programmer violates specification of the HLL.

3. What are the types of translators?

- Compiler
- Assembler
- Interpreter
- Pre-prossessor

4. Define system software. **(MAY 2012, 2015, 2018) (NOV 2012, 2016)**

System software consists of variety of programs that supports the operation of the computer. This software makes it possible for the user to focus on the other problems to be solved without needing to know how the machine works internally.

Example: operating system, assembler, and loader.

5. What are the basic components of the system software?

- Operating system
- Compiler
- Assembler
- Macro processor
- Loader or linker
- Debugger
- Text editor
- Database management
- Software engineering tools

6. Define Operating systems.

Operating system acts as an interface between the user of a computer and the computer hardware. The operating system is the most important program that runs on a computer. Every general-purpose computer must have an operating system to run other programs.

Example: Windows, Linux, UNIX, Dos

7. Difference between Application and System Software. (MAY 2017)

Application Software	System Software
Application software is computer software designed to help the user to perform specific tasks.	System software is computer software designed to operate the computer hardware and to provide a platform for running application software.
It is specific purpose software.	It is general-purpose software.
It executes as and when required.	It executes all the time in computer.
Application is not essential for a computer.	System software is essential for a computer.
Example: compiler, loader	**Example:** payroll, banking

8. Give some applications of operating system?

- to make the computer easier to use
- Muti-tasking ,Multi-Programming
- Parallel Processing
- Spooling
- Buffering
- to manage the resources in computer
- process management
- data and memory management
- to provide security to the user

9. Define Assembler? (MAY 2012)

- An assembler is a type of computer program that interprets software programs written in assembly language into machine language, code and instructions that can be executed by a computer.
- An assembler enables software and application developers to access, operate and manage a computer's hardware architecture and components.
- An assembler is sometimes referred to as the compiler of assembly language. It also provides the services of an interpreter.

10. Define interpreter. (NOV 2011) (NOV 2012)

- Interpreter is a set of programs which converts high level language program to machine language program line by line.
- It can immediately execute high-level programs. For this reason, interpreters are sometimes used during the development of a program, when a programmer wants to add small sections at a time and test them quickly.
- BASIC and LISP are especially designed to be executed by an interpreter. In addition, page description languages, such as PostScript, use an interpreter.

11. State the difference between assembler and Interpreter? (NOV 2013)

Assembler	Interpreter
An assembler is a type of computer program that interprets software programs written in assembly language into machine language, code and instructions that can be executed by a computer.	Interpreter is a set of programs which converts high level language program to machine language program line by line.
An assembler is sometimes referred to as the compiler of assembly language. It also provides the services of an interpreter.	BASIC and LISP are especially designed to be executed by an interpreter. In addition, page description languages, such as PostScript, use an interpreter.

12. How could literals be implemented in one pass assembler? (MAY 2013)

- Each literal operand is recognized during pass1, the assembler searches LITTAB for the specified literal name or value.
- If the literal is already present in the table, no action is needed; if it is not present, the literal is added to LITTAB.
- When Pass1 encounters a LTORG statement or the end of the program, the assembler makes a scan of the literal table.

13. What is SIC machine? (NOV 2014)

Simplified Instructional Computer (SIC) machine is a hypothetical computer that has been carefully designed to include the hardware features most often found on real machines, while avoiding unusual or irrelevant complexities.

14. What are the different Machine Architectures?

There are two versions of Simplified Instructional Computer (SIC) machines Architectures are

- Standard model (SIC)
- XE version (SIC/XE)

  ("XE" stands for eXtra Equipment or eXtra Expensive).

15. List the SIC machine architecture fields?

SIC machine consists of

- Memory
- Registers
- Data Formats
- Instruction Formats
- Addressing Modes
- Instruction Set
- Input and Output

16. What are the different registers in SIC Architecture?

Mnemonic	Number	Special use
A	0	Accumulator; used for arithmetic operations
X	1	Index register; used for addressing
L	2	Linkage register; jump to subroutine (JSUB) instruction stores the return address in this register
PC	8	Program Counter (PC); contains the address of the next instruction to be fetched for execution
SW	9	Status word; contains a variety of information, including a Condition Code (CC)

17. What are the types of Addressing modes in SIC?

- Direct addressing mode
- Indexed addressing mode

Mode	Indication	Target address calculation
Direct	X=0	TA=address
Indexed	X=1	TA=address+(X)

18. Define Instruction set. **(MAY 2013)**

An **instruction set**, or **instruction set architecture** (**ISA**), is the part of the computer architecture related to programming, including the native data types, instructions, registers, addressing modes, memory architecture, interrupt and exception handling, and external I/O.

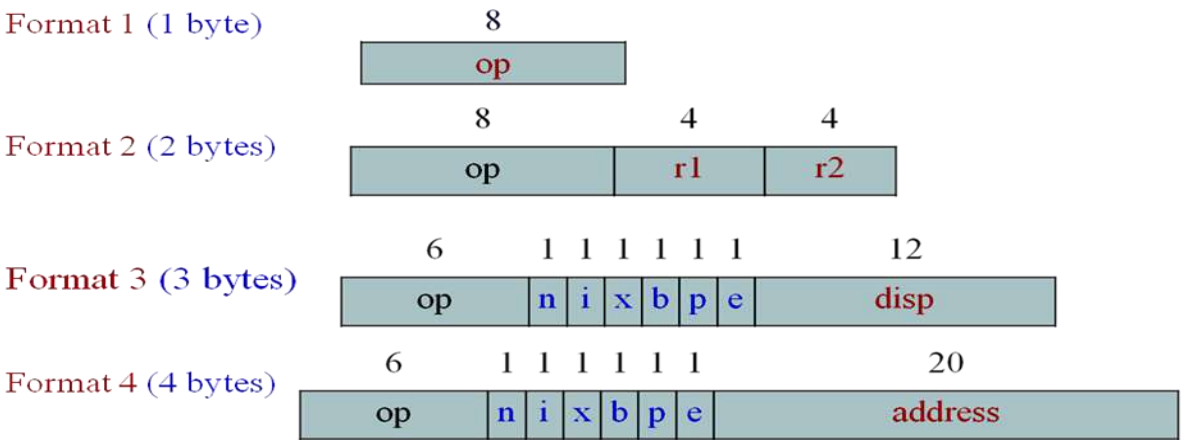
19. What are the types of Instruction set in SIC?

- Load and store registers: LDA, LDX, STA, STX, etc.
- Integer arithmetic Instructions: ADD, SUB, MUL, DIV, etc.
- Comparison Instructions: COMP
- COMP compares the value in register A with a word in memory, this instruction sets a Condition Code (CC) to indicate the result (<, =,>).
- Conditional jump instructions: JLT, JEQ, JGT
- Subroutine linkage: Jumps to the Subroutine (JSUB), RSUB

20. What are the different registers in SIC/XE Architecture?

Mnemonic	Number	Special use
B	3	Base register; used for addressing
S	4	General working register
T	5	General working register
F	6	Floating-point accumulator which is 48 bits

21. What are the different instruction formats in SIC/XE?



22. What are the types of addressing modes in SIC/XE?

- Base relative addressing mode
- Program Counter (PC) relative addressing mode
- direct addressing mode
- Indexed addressing mode
- immediate addressing mode
- indirect addressing mode

23. Differences between base relative and program counter relative addressing used in SIC/XE.

Mode	Indication	Target address calculation
Base relative	b=1,p=0	TA=(B)+ disp (0 ≤ disp ≤ 4095)
Program Counter (PC) relative	b=0,p=1	TA=(PC)+ disp (-2048 ≤ disp ≤2047)

24. Define indirect addressing?

- In indirect addressing mode the value i=0, n=1:
- The word at the TA is fetched
- Value in this word is taken as the address of the operand value

Example: ADD R5, [600]

Here the second operand is given in indirect addressing mode. First the word in memory location 600 is fetched and which will give the address of the operand.

25. Define immediate addressing.

- In this addressing mode the operand value is given directly. There is no need to refer memory. The immediate addressing is indicated by the prefix '#'.
- In case i = 1, n = 0 the address itself is the operand, no memory reference

Example: ADD #5

In this instruction one operand is in accumulator and the second operand is an immediate value the value 5 is directly added with the accumulator content and the result is stored in accumulator.

26. List the instruction sets in SIC/XE?

- Load and store the new registers: LDB, STB, etc.
- Floating-point arithmetic operations: ADDF, SUBF, MULF, DIVF
- Register MOve: RMO
- Register-to-register arithmetic operations: ADDR, SUBR, MULR, DIVR
- Supervisor call(SVC)

27. Write the program for BETA = ALPHA + INCR -1 and DELTA=GAMMA + INCR -1 using SIC instructions.

```
                LDA     ALPHA     LOAD ALPHA INTO REGISTER A
                ADD     INCR      ADD THE VALUE OF INCR
                SUB     ONE       SUBTRACT 1
                STA     BETA      STORE IN BETA
                LDA     GAMMA     LOAD GAMMA INTO REGISTER A
                ADD     INCR      ADD THE VALUE OF INCR
                SUB     ONE       SUBTRACT 1
                STA     DELTA     STORE IN DELTA
                ..... ........
ONE             WORD    1         ONE WORD CONSTANT
ALPHA           RESW    1         ONE WORD VARIABLES
BETA            RESW    1
GAMMA           RESW    1
DELTA           RESW    1
INCR            RESW    1
```

28. Write the program for BETA = ALPHA + INCR+1 and DELTA=GAMMA + INCR -1 using SIC /XE instructions.

```
                LDS     INCR      LOAD VALUE OF INCR INTO REGISTER S
                LDA     ALPHA     LOAD ALPHA INTO REGISTER A
                ADDR    S, A      ADD THE VALUE OF INCR
                SUB     #1        SUBTRACT 1
                STA     BETA      STORE IN BETA
                LDA     GAMMA     LOAD GAMMA INTO REGISTER A
                ADDR    S, A      ADD THE VALUE OF INCR
                SUB     #1        SUBTRACT 1
                STA     DELTA     STORE IN DELTA
                ..... ........
ALPHA           RESW    1         ONE WORD VARIABLES
BETA            RESW    1
GAMMA           RESW    1
DELTA           RESW    1
INCR            RESW    1
```

29. What are the fields available in an assembly language instruction? (NOV 2011)

The four fields available in an assembly language instruction are

- Label
- Opcode
- Operand
- Comments

30. What are the different assembler directives? (NOV 2012)

- START - specify program name and starting address of the program.
- END - indicate the end of the source program.
- BYTE - generate character or hexadecimal constants.
- WORD - Generate one word integer constant.
- RESB - Reserve the indicated no. of bytes for a data area.
- RESW - Reserve the indicated no. of words for a data area.

31. Define the basic functions of assembler. (NOV 2014) (MAY 2016)

1. Convert mnemonic operation codes to their machine language equivalents.
2. Convert symbolic operands to their equivalent machine addresses.
3. Build the machine instructions in the proper format.
4. Convert the data constants specified in the source program into internal machine representations.
5. Write the object program and the assembly listing.

32. What are the functions of two pass assembler?

Functions of Two Pass Assembler

Pass 1 - define symbols (assign addresses)

- Assign addresses to all statements in the program.
- Save the values assigned to all labels for use in Pass 2.
- Perform some processing assembler directives.

Pass 2 - assemble instructions and generate object program

- Assemble instructions
- Generate data values defined by BYTE, WORD, etc.
- Perform some processing assembler directives not done in Pass 1.
- Write the object program and the assembly listing.

33. What is the use of OPTAB? (MAY 2012)

- The Operation code table (OPTAB) contains the mnemonic operation code and its machine language equivalent.
- Some assemblers it may also contain information about instruction format and length.
- OPTAB is usually organized as a hash table, with mnemonic operation code as the key.

34. What is the use of SYMTAB? **(MAY 2012)**

- Symbol table (SYMTAB) - is used to store values (addresses) assigned to labels.
- It includes the name and value for each symbol in the source program, together with flags to indicate error conditions.
- Sometimes it may contain details about the data area.
- SYMTAB is usually organized as a hash table for efficiency of insertion and retrieval.

35. What are forward references?

It is a reference to a label that is defined later in a program.

Consider the statement

```
10      1000    STL         RETADR
....
....
80      1036    RETADR      RESW
```

The first instruction contains a forward reference RETADR. If we attempt to translate the program line by line, we will unable to process the statement in line10 because we do not know the address that will be assigned to RETADR .The address is assigned later in the program.

36. What is the format of the Object Program generated by the Assembler?

It contains 3 types of records:

Header record:

Col. 1	H
Col. 2~7	Program name
Col. 8~13	Starting address of object program (hexadecimal)
Col. 14-19	Length of object program in bytes (hexadecimal)

Text record

Col.1	T
Col. 2~7	Starting address for object code in this record (hex)
Col. 8~9	Length of object code in this record in bytes (hex)
Col.10~69	Object code (69-10+1)/6=10 instructions

End record

Col.1	E
Col.2~7	Address of first executable instruction in object program (hex)

37. List the features of machine dependent Assembler. **(NOV 2013)**

The features of machine dependent Assembler is

1. Instruction formats
2. Addressing modes and
3. Program Relocation

38. Define relocatable program.

An object program that contains the information necessary to perform required modification in the object code depends on the starting location of the program during load time is known as relocatable program.

39. Define modification record and give its format.

In Modification record contains the information about the modification in the object code during program relocation.

The general format is

Col 1 M

Col 2-7 Starting location of the address field to be modified relative to the beginning of the program.

Col 8-9 Length of the address field to be modified in half bytes.

40. What are literals? (NOV 2015)

It is often convenient for the programmer to be able to write the value of a constant operand as a part of the instruction that uses it. This avoids having to define the constant elsewhere in the program and makeup a label for it. Such an operand is called a ***literal*** because the value is stated "literally" in the instruction.

41. Define Literal pools. (MAY 2014)

All of the literal operands used in a program are gathered together into one or more literal pools. Normally literals are placed into pool at the end of the program.

42. What are the symbol defining statements generally used in assemblers?

- **'EQU'** - it allows the programmer to define symbols and specify their values directly.

  The general format is

  Symbol EQU value

- **'ORG'** - it is used to indirectly assign values to symbols. When this statement is encountered the assembler resets its location counter to the specified value.

  The general format is

  ORG value

43. Differentiate absolute expression and relative expression.

- If the result of the expression is an absolute value (constant) then it is known as absolute expression.

  Eg: BUFEND – BUFFER

- If the result of the expression is relative to the beginning of the program then it is known as relative expression. Label on instructions and data areas and references to the location counter values are relative terms.

  Eg: BUFEND + BUFFER

44. Define Program blocks.

The program blocks refer to segments of code that are rearranged within a single object program unit.

45. What are the types of program blocks?

The three blocks are

1. Default – it contains the executable instructions of the program.
2. CDATA – it contains all data areas that are a few words or less in length.
3. CBLKS – it contains all data areas that consist of larger block memory.

46. Define control section.

A control section is a part of the program that maintains its identity after assembly; each control section can be loaded and relocated independently of the others. Different control sections are most often used for subroutines or logical subdivisions of a program. The programmer can assemble, load and manipulate each of these control sections separately. The resulting flexibility is a major benefit of these control sections.

47. What is program linking?

- When control sections from logically related parts of a program, it is necessary to provide means for linking them together.
- For example, instructions in one control section might need to refer to instructions or data located in another section.

48. What is meant by external references? (NOV 2014)

In assembler where any other control section will be located at execution time. Such references between controls are called external references. The assembler generates information for each external reference that will allow the loader to perform the required linking.

49. What are the types of external symbols?

The two assembler directives to identify such references

1. External Definition (EXTDEF) names external symbols that are defined in a particular control section and may be used in other sections.
2. External References (EXTREF) names external symbols that are referred in a particular control section and defined in another control section.

50. Write the Format for Define and Refer record?

DERFINE RECORD

Col 1	D
Col 2-7	Name of external symbol defined in this control section
Col 8-13	Relative address of symbol within this control section
Col 14-73	Repeat information in col 2-13 for other external symbol

REFER RECORD

Col 1	R
Col 2-7	Name of external symbol defined in this control section
Col 8-73	Name of other external reference symbol

51. What is the use of load and go assembler?

- One pass assembler that generates their object code in memory for immediate execution is known as load and go assembler. No object programmer is written out and no loader is needed.
- It is useful in a system that is oriented toward program development and testing.

52. What is one pass assembler?

- One-pass assembler that generates their object code in memory for immediate execution.
- One-pass assemblers are used when it is necessary or desirable to avoid a second pass over the source program the external storage for the intermediate file between two passes is slow or is inconvenient to use.

53. State the limitations of one pass assembler? (MAY 2015)

Main problem: forward references to both data and instructions. One simple way to eliminate this problem: require that all areas be defined before they are referenced.

- It is possible, although inconvenient, to do so for data items.
- Forward jump to instruction items cannot be easily eliminated.

54. What is multi-pass assembler?

Prohibiting forward references in symbol definition:

- This restriction is not a serious inconvenience.
- Forward references tend to create difficulty for a person reading the program.

Allowing forward references

- To provide more flexibility.

55. Compare Assembler and Compiler. (NOV 2015)

Assembler	Compiler
Assembler is a computer program that reads code written in one language which is called Assembly language and translates into machine language.	Compiler is a computer program that reads code (like C, C++ code) and translates into machine language.
Assembler is used in low-level Assembly language.	Compiler is used in high-level language.
Assembler is less popular than compiler.	Compiler is more popular than Assembler.

56. What is meant by machine independent assembler features? (MAY 2016)

The machine independent assembler features are

1. Literals
2. Symbol defining statements
3. Expressions
4. Program blocks
5. Control Sections and Program Linking

57. What is the difference between the instructions LDA #3 and LDA THREE? (MAY 2016)

- In the first instruction immediate addressing is used. Here the value 3 is directly loaded into the accumulator register.
- In second instruction the memory reference is used. Here the address (address assigned for the symbol THREE) is loaded into the accumulator register.

58. List out the internal data structures used in assemblers? (NOV 2016)

Our simple assembler uses two major internal data structures are

1. Operation Code table (OPTAB) and
2. Symbol Table (SYMTAB)

- OPTAB is used to look up mnemonic operation codes and translate them to their machine language equivalents.
- SYMTAB is used to store values (addresses) assigned to labels.

59. List out the roles of Operating System in program execution? (MAY 2017)

- Hiding the complexities of hardware from the user.
- Managing between the hardware's resources which include the processors, memory, data storage and I/O devices.
- Handling interrupts generate by the I/O controllers.
- Sharing of I/O between many programs using the CPU.

60. Differentiate direct and indirect addressing Modes. (NOV 2017)

Direct addressing mode

- In direct addressing for formats 3 and 4 if b=0, p=0, x=0
- TA = Address field (0 ≤ disp ≤ 4095).
- Ex: LDA LENGTH

Indirect Addressing mode

- Set bit i=0, n=1, x=0
- The operand value points to an address that holds the address for the operand value.
- Ex: 002A J @ RETADDR

Direct addressing mode	Indirect Addressing mode
If the Address part has the address of an operand, then the instruction is said to direct address.	If the address bits of the instruction code are used as an actual operand is termed as indirect addressing.
Limited address space	Large address space
Target Address = Address field	Target Address = Address of Operand
Faster memory access	Usually slower than direct addressing

61. What is the purpose of program counter? **(NOV 2017)**

- Program Counter (PC); contains the address of the next instruction to be fetched for execution.
- A program counter is a register in a computer processor that contains the address (location) of the instruction being executed at the current time.

62. Differentiate between assembler and compiler. **(MAY 2018)**

Assembler	Compiler
Generates the relocatable machine code.	Generates the assembly language code or directly the executable code.
Assembly language code.	Preprocessed source code.
Assembler makes two passes over the given input.	The compilation phases are lexical analyzer, syntax analyzer, semantic analyzer, intermediate code generation, code optimization, code generation.
The relocatable machine code generated by an assembler is represented by binary code.	The assembly code generated by the compiler is a mnemonic version of machine code.

11 MARKS

1. Explain the system software and Machine structures? (11 MARKS)

System software consists of a variety of programs that support the operation of a computer. Application software focuses on an application or problem to be solved. System software consists of a variety of programs that support the operation of a computer.

Examples for system software are Operating system, compiler, assembler, macro processor, loader or linker, debugger, text editor, database management systems and software engineering tools. These software's make it possible for the user to focus on an application or other problem to be solved, without needing to know the details of how the machine works internally.

Types of software

1. Application software
2. System software

One characteristic in which most system software differs from application software is machine dependency.

- System software supports operation and use of computer.
- Application software provides solution to a problem.

Assembler translates mnemonic instructions into machine code. The instruction formats, addressing modes etc., are of direct concern in assembler design. Similarly, Compilers must generate machine language code, taking into account such hardware characteristics as the number and type of registers and the machine instructions available. Operating systems are directly concerned with the management of nearly all of the resources of a computing system.

There are aspects of system software that do not directly depend upon the type of computing system, general design and logic of an assembler, general design and logic of a compiler and code optimization techniques, which are independent of target machines. Likewise, the process of linking together independently assembled subprograms does not usually depend on the computer being used.

Components of system software

The language translator and the operating system are themselves programs. Their function is to get the user's program, which is written in a programming language, to run on the computer system. The programs which help in the execution of a user program are called **System Programs (SPs).** The collection of such SPs is the "system software" of a particular computer system. An identical argument holds in the case of a computer system. By writing a program in a higher level programming language, a programmer obviates the need to acquire totally new skills merely in order to run his program. The compiler performs the task of making his program understandable to the CPU.

Two fundamental aspects of the system are

- Making available new/better facilities
- Achieving efficient performance

The basic components of the system software are,

1. Interpreter
2. Assembler
3. Macro processing
4. Linking loader
5. Absolute loader
6. Operating system
7. Compiler
8. Debugging aids
9. Text editors
10. Utilities

Interpreter

This is also a translator converts the high level language into low level language. It converts the program line by line at a time. It analyses the source program statement by statement and it carries out the actions implied by each statement.

Assembler

It is a type of translator that converts assemble level language into machine level language. An assembler is used to convert the given mnemonics into numeric and converts them to executable or command files. Output is an object program plus program information that enables the loader to prepare the object program for execution.

Preprocessor

It is a translator that coverts one high level language into another high level language.

Macro processor

A macro call is an abbreviation for some codes. A macro definition is a sequence of code that has a name. A macro processor is a program that substitutes and specifies macro definition for macro calls.

Loader

Loader loads the object program and prepare for its execution. Loader schemes are absolute, relocating and direct linking. In general loader must load, relocate and link the object program.

Operating system

An operating system (OS) is a software program that manages the hardware and software resources of a computer. The OS performs basic tasks, such as controlling and allocating memory, prioritizing the processing of instructions, controlling input and output devices facilitating networking, and managing files.

Compiler

Compiler is a computer program (or set of programs) that translates text written in a computer language (the source language) into another computer language (the target language). The original sequence is usually called the source code and the output called object code. Commonly the output has a form suitable for processing by other programs (e.g., a linker), but it may be a human readable text file.

System Software and Architecture

- Machine dependency of system software
- System programs are intended to support the operation and use of the computer.

  Machine architecture differs in:
  - Machine code
  - Memory
  - Instruction formats
  - Addressing modes
  - Registers
- Machine independency of system software

  General design and logic is basically the same:
  - Code optimization
  - Subprogram linking

2. Write a short note on Assembler? (5 MARKS)

- An **Assembler** translates a program written in an assembly language to it machine language equivalent.
- An assembler is a type of computer program that interprets software programs written in assembly language into machine language, code and instructions that can be executed by a computer.
- An assembler enables software and application developers to access, operate and manage a computer's hardware architecture and components.
- An assembler is sometimes referred to as the compiler of assembly language. It also provides the services of an interpreter.
- An assembler is a program that accepts an assembly language program as input and produces its machine language equivalent along with information for the loader.
- Assembly language is converted into executable machine code by a utility program referred to as an assembler; the conversion process is referred to as assembly, or assembling the code.
- Assembly language uses a mnemonic to represent each low-level machine instruction or operation. Typical operations require one or more operands in order to form a complete instruction, and most assemblers can therefore take labels, symbols and expressions as operands to represent addresses and other constants, freeing the programmer from tedious manual calculations.
- **Macro assemblers** include a macroinstruction facility so that (parameterized) assembly language text can be represented by a name, and that name can be used to insert the expanded text into other code. Many assemblers offer additional mechanisms to facilitate program development, to control the assembly process, and to aid debugging.

Number of passes

There are two types of assemblers based on how many passes through the source are needed to produce the executable program.

- One-pass assemblers go through the source code once. Any symbol used before it is defined will require "errata" at the end of the object code telling the linker or the loader to "go back" and overwrite a placeholder which had been left where the as yet undefined symbol was used.
- Multi-pass assemblers create a table with all symbols and their values in the first passes, then use the table in later passes to generate code.

High-level assemblers

More sophisticated high-level assemblers provide language abstractions such as:

- Advanced control structures.
- High-level procedure/function declarations and invocations.
- High-level abstract data types, including structures/records, unions, classes, and sets.
- Sophisticated macro processing.
- Object-oriented programming features such as classes, objects, abstraction, polymorphism, and inheritance.

Basic elements

There is a large degree of diversity in the way the authors of assemblers categorize statements and in the nomenclature that they use. In particular, some describe anything other than a machine mnemonic or extended mnemonic as a pseudo-operation.

A typical assembly language consists of 3 types of instruction statements that are used to define program operations:

- Opcode mnemonics
- Data directives
- Assembly directives

Typical applications

- Assembly language is typically used in a system's boot code, (BIOS on IBM-compatible PC systems).
- computer cartridge games
- Microcontrollers (automobiles, industrial plants...)
- telecommunication equipment
- device drivers
- Very fast and compact but processor-specific.

3. Write a short note on Interpreter? **(5 MARKS)**

- ***Interpreter*** is also a translator converts the high level language into low level language. It converts the program line by line at a time.
- It analyses the source program statement by statement and it carries out the actions implied by each statement.
- It can immediately execute high-level programs. For this reason, interpreters are sometimes used during the development of a program, when a programmer wants to add small sections at a time and test them quickly.
- BASIC and LISP are especially designed to be executed by an interpreter. In addition, page description languages, such as PostScript, use an interpreter.

An ***interpreter*** is a computer program that directly executes, i.e. performs, instructions written in a programming or scripting language, without previously batch-compiling them into machine language. An interpreter generally uses one of the following strategies for program execution:

1. parse the source code and perform its behavior directly
2. translate source code into some efficient intermediate representation and immediately execute
3. explicitly execute stored precompiled code made by a compiler which is part of the interpreter system.

Early versions of the Lisp programming language and Dartmouth BASIC would be examples of the first type. Perl, Python, MATLAB, and Ruby are examples of the second, while UCSD Pascal is an example of the third type. Source programs are compiled ahead of time and stored as machine independent code, which is then linked at run-time and executed by an interpreter and compiler. Some systems, such as Smalltalk, contemporary versions of BASIC and Java.

While interpretation and compilation are the two main means by which programming languages are implemented, they are not mutually exclusive, as most interpreting systems also perform some translation work, just like compilers. The terms "interpreted language" or "compiled language" signify that the canonical implementation of that language is an interpreter or a compiler, respectively. A high level language is ideally an abstraction independent of particular implementations.

Compiler

Pros

- Less space
- Fast execution

Cons

- Slow processing
- Partly Solved (Separate compilation)
- Debugging
- Improved thru IDEs

Interpreter

Pros

- Easy debugging
- Fast Development

Cons

- Not for large projects
- Exceptions: Perl, Python
- Requires more space
- Slower execution
- Interpreter in memory all the time

4. Write a short note on Operating system? (OR) Give some applications of operating system.

(6 MARKS) (MAY 2016)

- An **operating system (OS)** is a collection of software that manages computer hardware resources and provides common services for computer programs. The operating system is an essential component of the system software in a computer system. Application programs usually require an operating system to function.
- **Operating system** acts as an interface between the user of a computer and the computer hardware.
- The operating system is the most important program that runs on a computer. Every general-purpose computer must have an operating system to run other programs.
- **Eg: Windows, Linux, UNIX, Dos**

Time-sharing operating systems schedule tasks for efficient use of the system and may also include accounting software for cost allocation of processor time, mass storage, printing, and other resources.

For hardware functions such as input and output and memory allocation, the operating system acts as an intermediary between programs and the computer hardware,[1][2] although the application code is usually executed directly by the hardware and will frequently make a system call to an OS function or be interrupted by it. Operating systems can be found on almost any device that contains a computer from cellular phones and video game consoles to supercomputers and web servers.

Examples of popular modern operating systems include Android, BSD, iOS, Linux, OS X, QNX, Microsoft Windows, Windows Phone, and IBM z/OS.

Types of operating systems

Real-time

A real-time operating system is a multitasking operating system that aims at executing real-time applications. Real-time operating systems often use specialized scheduling algorithms so that they can achieve a deterministic nature of behavior. The main objective of real-time operating systems is their quick and predictable response to events. They have an event-driven or time-sharing design and often aspects of both. An event-driven system switches between tasks based on their priorities or external events while time-sharing operating systems switch tasks based on clock interrupts.

Multi-user

A multi-user operating system allows multiple users to access a computer system at the same time. Time-sharing systems and Internet servers can be classified as multi-user systems as they enable multiple-user access to a computer through the sharing of time. Single-user operating systems have only one user but may allow multiple programs to run at the same time.

Multi-tasking vs. single-tasking

A multi-tasking operating system allows more than one program to be running at the same time, from the point of view of human time scales. A single-tasking system has only one running program. Multi-tasking can be of two types: pre-emptive and co-operative. In pre-emptive multitasking, the operating system slices the CPU time and dedicates one slot to each of the programs. In 16-bit versions of Microsoft Windows used cooperative multi-tasking. 32-bit versions of both Windows NT and Win9x used pre-emptive multi-tasking. Mac OS prior to OS X used to support cooperative multitasking.

Distributed System

A distributed operating system manages a group of independent computers and makes them appear to be a single computer. The development of networked computers that could be linked and communicate with each other gave rise to distributed computing. Distributed computations are carried out on more than one machine. When computers in a group work in cooperation, they make a distributed system.

Templated

In an OS, distributed and cloud computing context, templating refers to creating a single virtual machine image as a guest operating system, then saving it as a tool for multiple running virtual machines. The technique is used both in virtualization and cloud computing management, and is common in large server warehouses.

Embedded

Embedded operating systems are designed to be used in embedded computer systems. They are designed to operate on small machines like PDAs with less autonomy. They are able to operate with a limited number of resources. They are very compact and extremely efficient by design. Windows CE and Minix 3 are some examples of embedded operating systems.

Components of Operating Systems

The components of an operating system all exist in order to make the different parts of a computer work together. All user software needs to go through the operating system in order to use any of the hardware, whether it is as simple as a mouse or keyboard or as complex as an Internet component.

- Kernel
- Program execution
- Interrupts
- Protected mode and Supervisor mode
- Memory management
- Virtual memory
- Multitasking
- Disk access and file systems
- Device drivers
- Networking
- Security
- User interface

Applications of Operating Systems

- Multi-tasking
- Multi-Programming
- Parallel Processing
- Spooling
- Buffering
- process management
- data and memory management
- to provide security to the user

5. Explain in detail about SIC Machine Architecture? (11 MARKS) (NOV 2011, 2013, 2014, 2016) (MAY 2013, 2014, 2018)

Simplified Instructional Computer (SIC) is a hypothetical computer that includes the hardware features most often found on real machines.

There are two versions of SIC are

- Standard model (SIC)
- XE version (SIC/XE) ("XE" eXtra Equipment or eXtra Expensive).

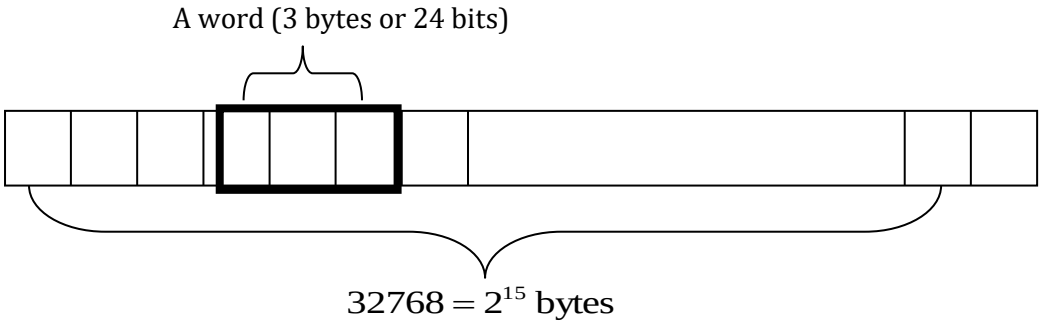
SIC Machine Architecture:

It consists of

1. Memory
2. Registers
3. Data Formats
4. Instruction Formats
5. Addressing Modes
6. Instruction Set
7. Input and Output

MEMORY

- It consists of 8-bit bytes.
- 3 consecutive bytes forms word (24 bits).
- All addresses on SIC are byte addresses.
- Words are addressed by the location of their lowest numbered byte
- 32,768 (2^{15}) bytes in the computer memory.



REGISTERS

- There are 5 registers.
- Each register is 24 bits in length.

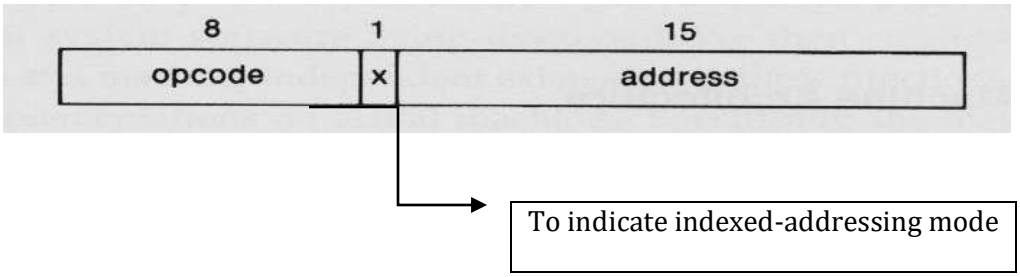
Mnemonic	Number	Special use
A	0	Accumulator; used for arithmetic operations
X	1	Index register; used for addressing
L	2	Linkage register; jump to subroutine (JSUB) instruction stores the return address in this register
PC	8	Program Counter (PC); contains the address of the next instruction to be fetched for execution
SW	9	Status word; contains a variety of information, including a Condition Code (CC)

DATA FORMATS

- Integers are stored as 24-bit binary numbers
- Characters :8-bit ASCII codes
- 2's complement for negative values
- There is no Floating-point numbers on Standard version SIC.

INSTRUCTION FORMATS

- All machine instructions on the standard SIC have 24-bit format.



ADDRESSING MODES

The two types of addressing modes are

1. Direct addressing mode
2. Indexed addressing mode
   - Set X bit to 0 or 1
   - Target address (TA) is calculated.

Mode	Indication	Target address calculation
Direct	X=0	TA=address
Indexed	X=1	TA=address+(X)

(X): Contents of a register or a memory location

INSTRUCTION SET

- Load and store registers: LDA, LDX, STA, STX, etc.
- Integer arithmetic Instructions: ADD, SUB, MUL, DIV, etc.
- All arithmetic operations involve register **A** and a word in memory, with the result being left in the register.
- Comparison Instructions : COMP
- COMP compares the value in register A with a word in memory, this instruction sets a condition code (CC) to indicate the result (<, =,>).
- Conditional jump instructions: JLT, JEQ, JGT
  - These instructions test the setting of CC and jump accordingly.
- Subroutine linkage: JSUB, RSUB
  - JSUB jumps to the subroutine, placing the return address in register L.
  - RSUB returns by jumping to the address contained in register L.

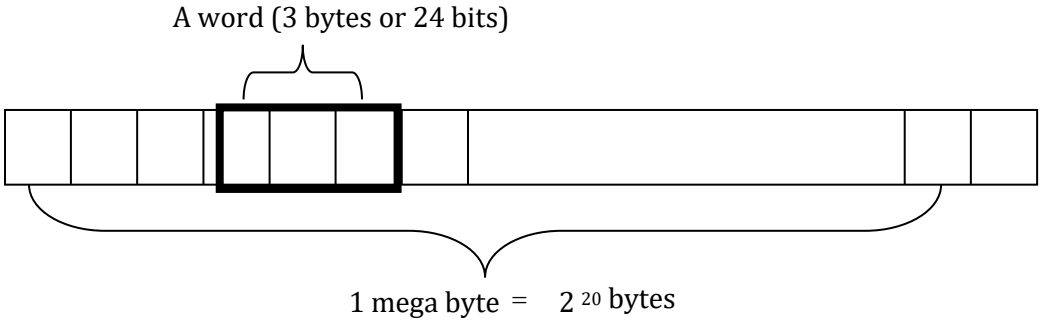
INPUT/OUTPUT

- Each IO device is assigned a unique 8-bit code.
- One byte at a time to or from the rightmost 8 bits of register A.
- Three instructions:
  1. Test device (TD)
     - Test whether the device is ready to send/receive
     - Test result is set in CC
  2. Read data (RD): read one byte from the device to register A.
  3. Write data (WD): write one byte from register A to the device.

6. Explain in detail about SIC/XE Machine Architecture? (OR) Explain the different addressing modes with suitable formats. (NOV 2015, 2017) (MAY 2017, 2018)

MEMORY

- Memory is same as SIC standard version.
- Maximum memory available on a SIC/XE system is 1 megabyte



REGISTERS

- Additional registers are provided by SIC/XE.

Mnemonic	Number	Special use
B	3	Base register –used for addressing
S	4	General working register-no special use
T	5	General working register-no special use
F	6	Floating-point accumulator (48 bits)

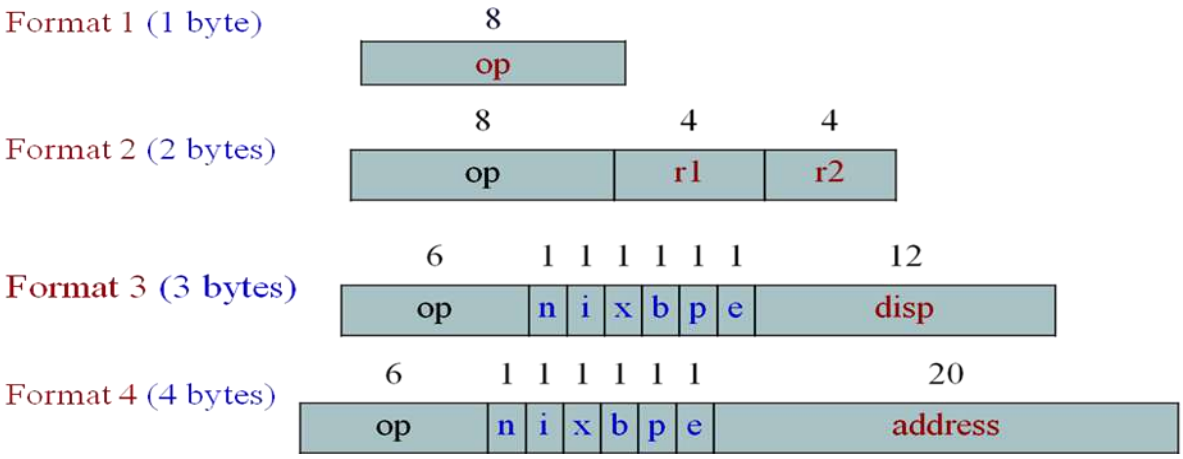
DATA FORMATS

- There is a 48-bit floating-point data type.
- Sign bit 0 (+ve) or 1 (-ve).
- exponent is an unsigned binary number between 0 and 2047.
- fraction is a value between 0 and 1.

1	11	36
S	**Exponent**	**fraction**

INSTRUCTION FORMATS

- Use relative addressing.
- Extend the address field to 20 bits.
- Instruction that reference memory uses.
- Formats 1 and 2 do not reference memory at all.
- Bit e distinguishes between format 3 and 4.



ADDRESSING MODES

1. Base relative addressing mode
2. Program Counter (PC) relative addressing mode
3. direct addressing mode
4. Indexed addressing mode
5. immediate addressing mode
6. indirect addressing mode

1. Base Relative addressing mode

- The displacement field in format 3 instruction is interpreted as a 12 bit unsigned integer.
- Ex: 1056 STX LENGTH

Mode	Indication	Target address calculation
Base relative	b=1,p=0	**TA = (B)+ disp** (0 ≤ disp ≤ 4095)
Program Counter (PC) relative	b=0,p=1	**TA =(PC)+ disp** (-2048 ≤ disp ≤ 2047)

2. Program counter Relative Addressing mode

- The displacement field in format 3 instruction is interpreted as a 12 bit signed integer.
- Negative values 2's complement.
- Ex: 0000 STL RETADDR

3. Direct addressing mode

- In direct addressing for formats 3 and 4 if b=0, p=0, x=0
- Ex: LDA LENGTH
- **TA = Address field (0 ≤ disp ≤ 4095).**

4. Indexed addressing mode

- Set bit x=1, b=1, p=0 value to be added to the value stored at the register x to obtain real address of the operand.
- STCH BUFFER , X
- **TA = (B) + (X) + disp**

5. Immediate addressing mode

- Set bit i=1, n=0: immediate addressing
- TA is used as the operand value, no memory reference
- Ex: LDA #9
- **TA = operand value = disp**

6. Indirect Addressing mode

- Set bit i=0, n=1, x=0
- The word at the TA is fetched value in this word is taken as the address of the operand value
- Ex: 002A J @ RETADDR

INSTRUCTION SET

- Load and store the new registers: LDB, STB, etc.
- Floating-point arithmetic operations- ADDF, SUBF, MULF, DIVF
- Register move: RMO
- Register-to-register arithmetic operations- ADDR, SUBR, MULR, DIVR
- Supervisor call: SVC-for generating an interrupt.

INPUT AND OUTPUT

- There are I/O channels that can be used to perform input and output while the CPU is executing other instructions.
- The instructions SIO, TIO, and HIO are used to start test and halt the operation of I/O channels.

7. Write a SIC and SIC/XE assembler programs? (11 MARKS) (MAY 2013)

DATA MOVEMENT OPERATION

ALPHA=5, C1=Z using SIC instructions

```
            LDA     FIVE    LOAD CONSTANT 5 INTO REGISTER A
            STA     ALPHA   STORE IN ALPHA
            LDCH    CHARZ   LOAD CHARACTER 'Z' INTO REGISTER A
            STCH    C1      STORE IN CHARACTER VARIABLE C1
            .......
ALPHA       RESW    1       ONE WORD VARIABLE
FIVE        WORD     5      ONE WORD CONSTANT
CHARZ       BYTE    C'Z'    ONE BYTE CONSTANT
C1          RESB    1       ONE BYTEVARIABLE
```

ALPHA=5, C1=Z using SIC/XE instructions

```
            LDA     #5      LOAD VALUE 5 INTO REGISTER A
            STA     ALPHA   STORE IN ALPHA
            LDA     #90     LOAD ASCII CODE FOR 'Z' INTO REGISTER A
            STCH    C1      STORE IN CHARACTER VARIABLE C1
            ......
ALPHA       RESW    1       ONE WORD VARIABLE
C1          RESB    1       ONE BYTEVARIABLE
```

ARITHMETIC OPERATION

BETA = ALPHA + INCR+1 and DELTA=GAMMA + INCR -1 using SIC instructions.

```
            LDA     ALPHA   LOAD ALPHA INTO REGISTER A
            ADD     INCR    ADD THE VALUE OF INCR
            SUB     ONE     SUBTRACT 1
            STA     BETA    STORE IN BETA
            LDA     GAMMA   LOAD GAMMA INTO REGISTER A
            ADD     INCR    ADD THE VALUE OF INCR
            SUB     ONE     SUBTRACT 1
            STA     DELTA   STORE IN DELTA
            ..... ........
ONE         WORD     1      ONE WORD CONSTANT
```

```
ALPHA   RESW   1   ONE WORD VARIABLES
BETA    RESW   1
GAMMA   RESW   1
DELTA   RESW   1
INCR    RESW   1
```

BETA = ALPHA + INCR+1 and DELTA=GAMMA + INCR -1 using SIC /XE instructions.

```
        LDS     INCR    LOAD VALUE OF INCR INTO REGISTER S
        LDA     ALPHA   LOAD ALPHA INTO REGISTER A
        ADDR    S, A    ADD THE VALUE OF INCR
        SUB     #1      SUBTRACT 1
        STA     BETA    STORE IN BETA
        LDA     GAMMA   LOAD GAMMA INTO REGISTER A
        ADDR    S, A    ADD THE VALUE OF INCR
        SUB     #1      SUBTRACT 1
        STA     DELTA   STORE IN DELTA
        …… ……..
ALPHA   RESW    1       ONE WORD VARIABLES
BETA    RESW    1
GAMMA   RESW    1
DELTA   RESW    1
INCR    RESW    1
```

8. Explain in detail about basic assembler functions? (11 MARKS) (NOV 2013) (MAY 2015)

- An assembler is a type of computer program that interprets software programs written in assembly language into machine language, code and instructions that can be executed by a computer.
- An assembler is sometimes referred to as the compiler of assembly language. It also provides the services of an interpreter.

Basic Assembler Functions:

The basic assembler functions are:

- Translating mnemonic operation codes to their machine language equivalents.
- Assigning machine addresses to symbolic labels.

Assembler Directives

The various basic assembler directives are

- START - specify program name and starting address of the program.
- END - indicate the end of the source program
- BYTE - generate character or hexadecimal constants.
- WORD - Generate one word integer constant.
- RESB - Reserve the indicated no. of bytes for a data area.
- RESW - Reserve the indicated no. of words for a data area.

Assembler's Functions

1. Convert mnemonic operation codes to their machine language equivalents.
2. Convert symbolic operands to their equivalent machine addresses.
3. Build the machine instructions in the proper format.
4. Convert the data constants specified in the source program into internal machine representations.
5. Write the object program and the assembly listing

Functions of Two Pass Assembler

The assembled program will be loaded into memory for execution.

Pass 1 - define symbols (assign addresses)

- Assign addresses to all statements in the program.
- Save the values assigned to all labels for use in Pass 2.
- Perform some processing assembler directives.

Pass 2 - assemble instructions and generate object program

- Assemble instructions.
- Generate data values defined by BYTE, WORD, etc.
- Perform some processing assembler directives not done in Pass 1.
- Write the object program and the assembly listing.

The simple object program contains three types of records:

1. Header record
2. Text record and
3. End record

- The header record contains the starting address and length.
- Text record contains the translated instructions and data of the program, together with an indication of the addresses where these are to be loaded.
- The end record marks the end of the object program and specifies the address where the execution is to begin.

Header record

Col. 1	H
Col. 2~7	Program name
Col. 8~13	Starting address of object program (hexadecimal)
Col. 14-19	Length of object program in bytes (hexadecimal)

Text record

Col.1	T
Col. 2~7	Starting address for object code in this record (hex)
Col. 8~9	Length of object code in this record in bytes (hex)
Col.10~69	Object code (69-10+1)/6=10 instructions

End record

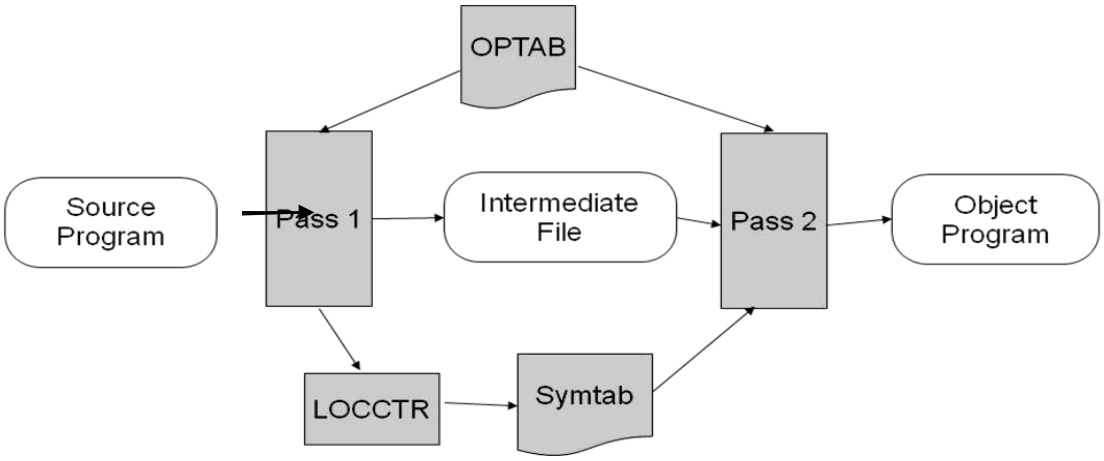
Col.1	E
Col.2~7	Address of first executable instruction in object program (hex)

9. Discuss about algorithms and data structure with an example? (11 MARKS)

Algorithms and Data structures

The simple assembler uses two major internal data structures:

1. Operation Code Table (OPTAB)
2. Symbol Table (SYMTAB)



OPTAB (Operation Code Table)

- It is used to lookup mnemonic operation codes and translates them to their machine language equivalents.
- In more complex assemblers the table also contains information about instruction format and length.
- OPTAB is usually organized as a hash table, with mnemonic operation code as the key.
- The hash table organization is particularly appropriate, since it provides fast retrieval with a minimum of searching.
- In pass 1 the OPTAB is used to look up and validate the operation code in the source program.
  In pass 2, it is used to translate the operation codes to machine language. In simple SIC machine this process can be performed in either in pass 1 or in pass 2.
- In pass 2 we take the information from OPTAB to tell us which instruction format to use in assembling the instruction, and any peculiarities of the object code instruction.

SYMTAB (Symbol Table)

- SYMTAB is used to store values (addresses) assigned to labels.
- This table includes the name and value for each label in the source program, together with flags to indicate the error conditions.

LOCCTR (Location Counter)

- Location Counter (LOCCTR) is initialized to the beginning address mentioned in the START statement of the program. After each statement is processed, the length of the assembled instruction is added to the LOCCTR to make it point to the next instruction.
- Whenever a label is encountered in an instruction the LOCCTR value gives the address to be associated with that label.

Pass 1 Algorithm

```
begin
        read first input line
        if OPCODE = 'START' then
        begin
                save #[Operand] as starting address
                initialize LOCCTR to starting address
                write line to intermediate file
                read next line
        end( if START)
else
        initialize LOCCTR to 0
while OPCODE != 'END' do
        begin
                if this is not a comment line then
                        begin
                                if there is a symbol in the LABEL field then
                                        begin
                                                search SYMTAB for LABEL
                                                if found then
                                                set error flag (duplicate symbol)
                                                else {if symbol}
                                        search OPTAB for OPCODE
                                        if found then
                                                add 3 (instr length) to LOCCTR
                                        else if OPCODE = 'WORD' then
                                                add 3 to LOCCTR
                                        else if OPCODE = 'RESW' then
                                                add 3 * #[OPERAND] to LOCCTR
                                        else if OPCODE = 'RESB' then
                                                        add #[OPERAND] to LOCCTR
```

```
                    else if OPCODE = 'BYTE' then
                    begin
                        find length of constant in bytes
                        add length to LOCCTR
                    end {if BYTE}
                            else
                                set error flag (invalid operation code)
                        end (if not a comment)
                    write line to intermediate file
                    read next input line
                end { while not END}
            write last line to intermediate file
        Save (LOCCTR – starting address) as program length
    end {Pass1}
```

- The algorithm scans the first statement START and saves the operand field (the address) as the starting address of the program. Initializes the LOCCTR value to this address. This line is written to the intermediate line.
- If no operand is mentioned the LOCCTR is initialized to zero. If a label is encountered, the symbol has to be entered in the symbol table along with its associated address value.
- If the symbol already exists that indicates an entry of the same symbol already exists. So an error flag is set indicating a duplication of the symbol.
- It next checks for the mnemonic code, it searches for this code in the OPTAB. If found then the length of the instruction is added to the LOCCTR to make it point to the next instruction.
- If the opcode is the directive WORD it adds a value 3 to the LOCCTR. If it is RESW, it needs to add the number of data word to the LOCCTR. If it is BYTE it adds a value one to the LOCCTR, if RESB it adds number of bytes.
- If it is END directive then it is the end of the program it finds the length of the program by evaluating current LOCCTR – the starting address mentioned in the operand field of the END directive. Each processed line is written to the intermediate file.

Pass 2 Algorithm

```
begin
    read first  input line {from intermediate file}
    if OPCODE = 'START' then
        begin
            write listing line
            read next input line
        end {if START}
```

```
        write Header record to object program
        initialize first Text record
        while OPCODE != 'END' do
            begin
                if this is not comment line then
                    begin
                        search OPTAB for OPCODE
                    if found then
                       begin
                          if there is a symbol in OPERAND field then
                            begin
                                search SYMTAB for OPERAND
                                if found then
                            begin
                                store symbol value as operand address
                                set error flag (undefined symbol)
                            end
                    end {if symbol}
                else
                    store 0 as operand address
                assemble the object code instruction
            end {if symbol}
        else if OPCODE = 'BYTE' or 'WORD" then
            convert constant to object code
        if object code doesn't fit into current Text record then
            begin
                Write text record to object code
                initialize new Text record
            end
            add object code to Text record
        end {if not comment}
      write listing line
            read next input line
        end {while not END}
    write last Text record to object program
    write End record to object program
    write last listing line
end {Pass 2}
```

- The first input line is read from the intermediate file. If the opcode is START, then this line is directly written to the list file. A header record is written in the object program which gives the starting address and the length of the program (which is calculated during pass 1). Then the first text record is initialized. Comment lines are ignored. In the instruction, for the opcode the OPTAB is searched to find the object code.
- If a symbol is there in the operand field, the symbol table is searched to get the address value for this which gets added to the object code of the opcode. If the address not found then zero value is stored as operands address. An error flag is set indicating it as undefined. If symbol itself is not found then store 0 as operand address and the object code instruction is assembled.
- If the opcode is BYTE or WORD, then the constant value is converted to its equivalent object code (for example, for character EOF, its equivalent hexadecimal value '454f46' is stored). If the object code cannot fit into the current text record, a new text record is created and the rest of the instructions object code is listed. The text records are written to the object program. Once the whole program is assemble and when the END directive is encountered, the End record is written.

EXAMPLE: SIC –Standard Version

The Source program is **P = X + Y + 2XY**

LOCCTR	LABEL	OPCODE	OPERAND	OBJECT CODE
2000	**SAMPLE**	**START**		**2000**
2000	LDA	X		**002018**
2003	ADD	Y		**18201B**
2006	STA	TEMP		**C0201E**
2009	LDA	X		**002018**
200C	MUL	Y		**20201B**
200F	MUL	TWO		**202024**
2012	ADD	TEMP		**18201E**
2015	STA	P		**0C2021**
2018	X	RESW	1	
201B	Y	RESW	1	
201E	TEMP	RESW	1	
2021	P	RESW	1	
2024	TWO	WORD	2	**000002**
2027	**END**			

The object program of this source program

H^**SAMPLE**^002000^000027

T^002000^**1B**^002018^18201B^0C201E^002018^20021B^202024^18201E^0C2021^000002

E^002000

SYMTAB

SYMBOL	VALUE
X	2018
Y	201B
TEMP	201E
P	2021
TWO	2024

OPTAB

MNEMONICS	OPERAND
LDA	00
ADD	18
STA	0C
MUL	20

10. Explain in detail about machine dependent assembler features? (11 MARKS) (MAY 2012)

Consider the design and implementation of an assembler for the more complex XE version of SIC.

- instruction formats
- addressing modes
- program relocation

(i)Instruction Format

Instructions can be:

- Instructions involving register to register
- Instructions with one operand in memory, the other in Accumulator
- Extended instruction format

Format 1

Length = 1 byte

8 bits

OPCODE

Format 2

Length = 2 bytes

It is used for register –to – register instruction

8 bits 4 4

OPCODE	r1	r2

Format 3

Length = 3 bytes

Register-memory instruction

6 1 1 1 1 1 1 12

OPCODE	n	i	x	b	p	e	Displacement

Displacement can be calculated in two ways

1. PC relative
2. Base relative

PC –relative displacement Calculation

TA= [PC] + disp (-2048 ≤ displacement ≤ 2047)

BASE – relative displacement Calculation

TA= (B) + disp (0 ≤ disp ≤ 4095)

Format 4 :(Extended format) (if disp>4095)

Extended format of the instruction must be indicated by the prefix (+)

6	1	1	1	1	1	1	20
OPCODE	n	i	x	b	p	e	Address

Translation

1. Register Translations for the Instruction involving Register-Register addressing mode

- Register name (A, X, L, B, S, T, F, PC, SW) and their values (0, 1, 2, 3, 4, 5, 6, 8, 9).
- **During pass 1** the registers can be entered as part of the symbol table itself. The value for these registers is their equivalent numeric codes.
- **During pass2,** these values are assembled along with the mnemonics object code. If required a separate table can be created with the register names and their equivalent numeric values.

2. Translation involving Register-Memory instructions

- In SIC/XE machine there are four instruction formats and five addressing modes.
- For formats and addressing modes
- The instruction formats, format-3 and format-4 instructions are Register-Memory type of instruction.
- One of the operand is always in a register and the other operand is in the memory.
- The addressing mode tells us the way in which the operand from the memory is to be fetched.

3. Address Translation

- Most register-memory instructions use program counter relative or base relative addressing
- Format 3: 12-bit disp field
  - Base-relative: 0~4095
  - PC-relative: -2048~2047
- Format 4: 20-bit address field

(ii) Addressing Modes

- PC-relative or Base-relative addressing op m
- Indirect addressing op @ m
- Immediate addressing op #c
- Extended format + op m
- Index addressing op m , x

Program-Counter Relative Addressing Mode

- In this usually format-3 instruction format is used. The instruction contains the opcode followed by a 12-bit displacement value.
- The range of displacement values are from 0 -2048. This displacement value is added to the current contents of the program counter to get the target address of the operand required by the instruction.

TA= [PC] + disp -2048 ≤ disp ≤ 2047

Base-Relative Addressing Mode

- In this mode the base register is used to mention the displacement value.
- Therefore the target address is

TA= (B) + disp 0 ≤ disp ≤ 4095

Immediate Addressing Mode

- In this mode no memory reference is involved. If immediate mode is used the target address is the operand itself.

Indirect and PC-relative mode

- In this type of instruction the symbol used in the instruction is the address of the location which contains the address of the operand. The address of this is found using PC-relative addressing mode.

(iii)Program Relocation:

- The assembler can identify for the loader those part of the object program that needed modification.
- An object program that contains the information necessary to perform this kind of modification is called **Relocatable program**.
- The instructions which are in the relative addressing modes (PC-relative or Base-relative) does not require any modification, wherever the program is loaded in the memory.

More than one program at a time sharing the memory and resources.

- Absolute program, starting address 1000

e.g. 55 101B LDA THREE 00102D

- Relocate the program to 2000

e.g. 55 101B LDA THREE 00202D

- Each Absolute address should be modified
- Modification record for each address field that needs to be changed when the program is relocated.

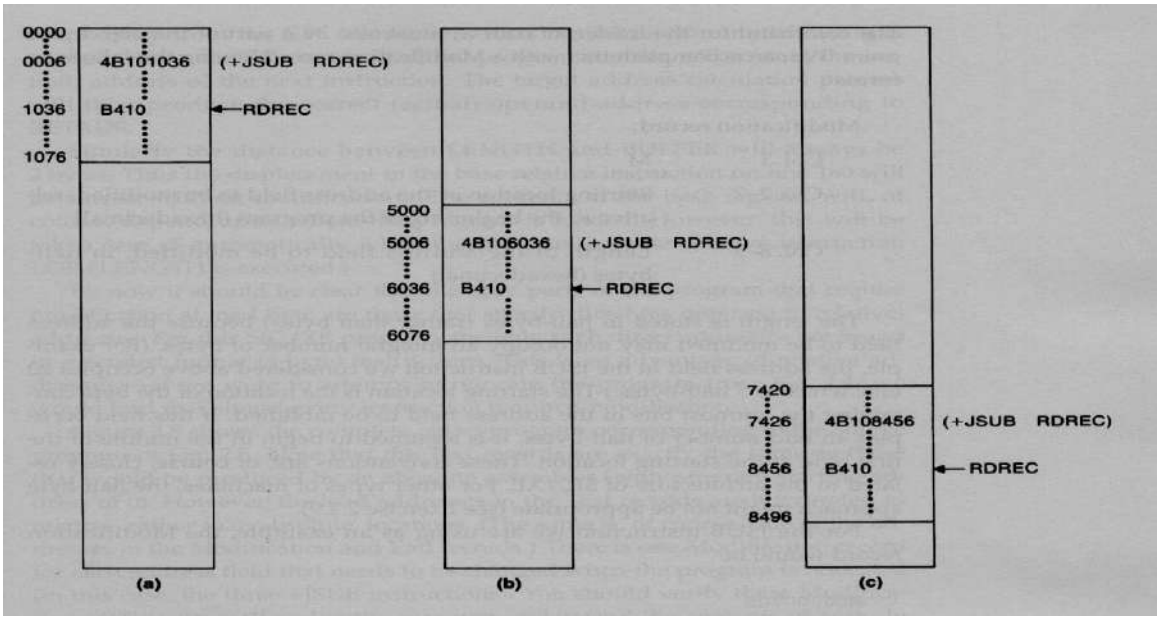
The format of Modification Record is

MODIFICATION RECORD

Col 1	M
Col 2-7	Starting location of the address field to be modified, relative to the beginning of the program (hexadecimal).
Col 8-9	Length of the address field to be modified in half bytes (hexadecimal).

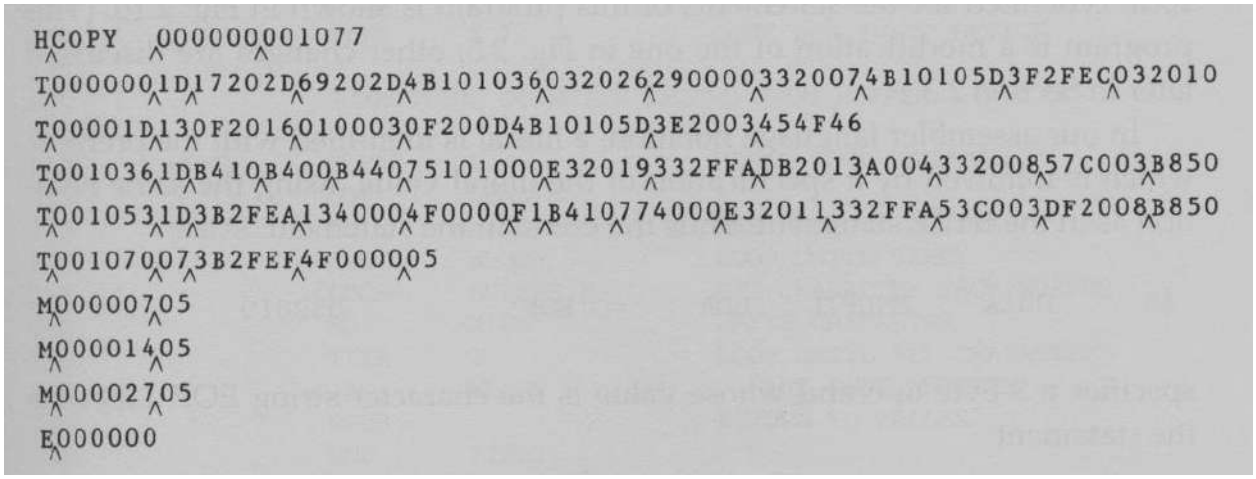
- One modification record for each address to be modified.
- The length is stored in half-bytes (20 bits = 5 half-bytes).
- The starting location is the location of the byte containing the leftmost bits of the address field to be modified.
- If the field contains an odd number of half-bytes, the starting location begins in the middle of the first byte.

Example of Program Relocation



For example, a program is loaded beginning at address 0000. The JSUB instruction is loaded at address 0006. The address field of this instruction contains 01036, which is the address of the instruction labeled RDREC.

Relocatable Object Code



```
HCOPY  000000001077
T0000001D17202D69202D4B10103603202629000033200 74B10105D3F2FEC032010
T00001D130F20160100030F200D4B10105D3E2003454F46
T001036 1DB410B400B44075101000E32019332FFADB2013A00433200857C003B850
T00105 31D3B2FEA1340004F0000F1B410774000E32011332FFA53C003DF2008B850
T001070073B2FEF4F000005
M00000705
M00001405
M00002705
E000000
```

11. Explain briefly about machine independent features of assembler? (11 MARKS)
(NOV 2011, 2012) (MAY 2014, 2015)

These are the features which do not depend on the architecture of the machine. These are:

1. Literals
2. Symbol Defining Statement
3. Expressions
4. Program Blocks
5. Control Sections and Program Linking

(i) LITERALS

- Literal is a constant operand which is used to write the value of it as a part of the instruction.
- This avoids having to define the constant elsewhere in the program and make up a label for it.
- A literal is defined with a prefix = followed by a specification of the literal value.

Example

001A	ENDFIL	LDA	**=C'EOF'**	032010
		LTORG		
002D	*	**=C'EOF'**		454F46
1062	WLOOP	TD	**=X'05'**	E32011

Difference between Literal &Immediate Addressing

- With immediate addressing, the operand value is assembled as part of the instruction itself.

  Eg: LDA #3 010003

- With literal, the assembler generates specified value as a constant at some other memory location the address of this generated constant is used as target address for the machine instruction.

  Eg: ENDFIL LDA =C'EOF' 032010

Literal Pool

- All the operands used in the program are gathered together into one or more literal pools.
- Normally literals are placed into a pool at the end of the program.
- In some cases it is desirable to place literals into a pool at some other location in the object program using the directive LTORG (LITERAL ORIGIN).
- When the assembler encounters the LTORG statement, it creates a literal pool that contains all of the literals operands used since the previous LTORG or the beginning of the program. Literals placed by LTORG will not be repeated at the pool at the end of the program .The duplicated literals are recognized by comparing the generated value of the constant operands.

Eg: usage of literals

```
LDA = C'EOF'
LDA = X'454F46'
```

		Object code
Definition of literals	*= C'EOF'	454F46
	*=C'454F46'	454F46

- Literals can be used to refer to the current value of the location counter (address of the memory location).

Eg:
```
base   *
LDB    =*
```

- The above instructions are used to specify the operand with value as current value of location counter.

(ii) SYMBOL DEFINING STATEMENT

EQU Statement

- Most assemblers provide an assembler directive that allows the programmer to define symbols and specify their values.
- The directive used for this **EQU** (Equate).

The general form of the statement is

Symbol EQU value

- This statement defines the given symbol (i.e., entering in the SYMTAB) and assigning to it the value specified.
- The value can be a constant or an expression involving constants and any other symbol which is already defined. One common usage is to define symbolic names that can be used to improve readability in place of numeric values.

For example

+LDT #4096

This loads the register T with immediate value 4096; this does not clearly what exactly this value indicates.

If a statement is included as:

MAXLEN	**EQU**	**4096** and then
	+LDT	**#MAXLEN**

Then it clearly indicates that the value of MAXLEN is some maximum length value. When the assembler encounters EQU statement, it enters the symbol MAXLEN along with its value in the symbol table.

During LDT the assembler searches the SYMTAB for its entry and its equivalent value as the operand in the instruction.

Another common usage of EQU statement is for defining values for the general-purpose registers. The assembler can use the mnemonics for register usage like a-register A, X – index register and so on. But there are some instructions which require numbers in place of names in the instructions. For example in the instruction RMO 0, 1 instead of RMO A, X.

The programmer can assign the numerical values to these registers using EQU directive

A	**EQU**	**0**
X	**EQU**	**1**

These statements will cause the symbols A, X, L… to be entered into the symbol table with their respective values. An instruction RMO A, X would then be allowed. As another usage if in a machine that has many general purpose registers named as R1, R2,…, some may be used as base register, some may be used as accumulator. Their usage may change from one program to another.

In this case we can define these requirement using EQU statements.

BASE	**EQU**	**R1**
INDEX	**EQU**	**R2**
COUNT	**EQU**	**R3**

One restriction with the usage of EQU is whatever symbol occurs in the right hand side of the EQU should be predefined. For example, the following statement is not valid:

BETA	**EQU**	**ALPHA**
ALPHA	**RESW**	**1**

As the symbol ALPHA is assigned to BETA before it is defined. The value of ALPHA is not known.

ORG Statement

- This directive can be used to indirectly assign values to the symbols. The directive is usually called ORG (for origin).
- Its general format is:

ORG value

Where value is a constant or an expression involving constants and previously defined symbols. When this statement is encountered during assembly of a program, the assembler resets its location counter (LOCCTR) to the specified value. Since the values of symbols used as labels from LOCCTR, the ORG

statement will affect the values of all labels defined until the next ORG. ORG can be useful in label definition.

Suppose we need to define a symbol table with the following structure:

SYMBOL	**6 Bytes**
VALUE	**3 Bytes**
FLAG	**2 Bytes**

For example, if the symbol table is defined in the following structure:

STAB (100 entries)

SYMBOL	VALUE	FLAGS

In this table, the SYMBOL field contains a 6 byte user defined symbol. VALUE is a one word representation of the value assigned to the symbol. FLAGS is a 2 byte field that specifies symbol type and other information.

(iii) EXPRESSIONS

- Assemblers also allow use of expressions in place of operands in the instruction. Each such expression must be evaluated to generate a single operand value or address.
- Assemblers generally arithmetic expressions formed according to the normal rules using arithmetic operators +, - *, /.Division is usually defined to produce an integer result.

Individual terms may be

- constants,
- user-defined symbols, or
- Special terms.

- The only special term used is * (the current value of location counter) which indicates the value of the next unassigned memory location.

Thus the statement

**BUFFEND EQU *

Hence, expressions are classified as either

1. Absolute expression or
2. Relative expressions

Absolute Expression

- The expression that uses only absolute terms is absolute expression.
- Absolute expression may contain relative term provided the relative terms occur in pairs with opposite signs for each pair.

Example

MAXLEN	**EQU**	**BUFEND-BUFFER**

- The expression can have only absolute terms.

Example

MAXLEN	**EQU**	**1000**

- But produce the absolute value 1000 which is the length of buffer

Relative Expression

- The value of relative expression is dependent of starting address of the program.
- It may contain relative terms or absolute terms but it must produce the value which is relative to the starting address of the program.

Example

SYMBOL	**EQU**	**STAB**	
VALUE	**EQU**	**STAB+6 →**	**1000+6=1006**
FLAGS	**EQU**	**STAB+9 →**	**1000+9=1009**

- The value of the expression is 1006 which is some memory location within the program and also relative to the beginning address of the program.

(iv) PROGRAM BLOCKS

- The program blocks refer to segments of code that are rearranged within a single object program unit.
- The control sections to refer to segments that are translated independent object program units.
- This feature allows more flexible handling of source and object programs.
- Here the generated machine instructions and data to appear in the object program in different order from the corresponding source statements.
- This results the creation of several independent parts of the object program.
- These parts maintain their identity and are handled separately by the loader.

Use Directives

- USE Directive indicates which portion of the source program belongs to the various blocks.
- If no USE statements are included, the entire program belongs to the single block.
- This directive also indicate a continuation of previously begins block.
- Each program block contains several separate segments of the source program.
- The assembler with rearrange these segments to gather together the pieces of each block.
- The assembler accomplishes this logical rearrangement of code by maintaining a separate location counter for each program block during pass1.

Implementation of Program Blocks

- The location counter for a block is initialized to 0 when the block is first begun.
- The current value of this location counter is saved when switching to another block and saved value is restored when resuming a previous block.

During Pass 1

- Each program block has a separate location counter.
- Each label is assigned an address that is relative to the start of the block that contains it.
- At the end of Pass 1, the latest value of the location counter for each block indicates the length of that block.
- The assembler can then assign to each block a starting address in the object program.

During Pass 2

- The address of each symbol can be computed by adding the assigned block starting address and the relative address of the symbol to that block.

For example, a program can be divided into 3 blocks namely:

1. **Default block** → This block contains executable instruction s of source program.
2. **CDATA** → This block includes storage area for few bytes.
3. **CBLCKS** → This block contains storage area for larger number of bytes.

- At the end of PASS1, the assembler constructs a table that contains the starting address and lengths of all blocks.

For example, it might be:

Block Name	Block No	Starting Address	Length
Default	0	0000	0066
CDATA	1	0066	000B
CBLCKS	2	0071	1000

- When the labels are entered in to the symbol table, the block name or number is stored along with its relative address.

Example symbol table

Block no.	Symbol	value
0	FIRST	0000
0	CLOOP	0003
0	ENDFIL	0015
1	RETADR	0000

1	LENGTH	0003
2	BUFFER	0000
2	BUFEND	1000

(v) CONTROL SECTIONS AND PROGRAM LINKING

- A **control section** is a part of the program that maintains its identity after assembly; each such control section can be loaded and relocated independently of others.
- Different control sections are most often used for subroutines or other logical subdivisions of a program.
- The programmer can assemble, load, and manipulate each of these control sections separately.
- The resulting flexibility is a major benefit of using control sections.
- When control sections form logically related parts of a program, it is necessary to provide some means for **linking** them together.
- For example, instructions in one control section might need to refer to instructions or data located in another section. Because control sections are independently loaded and relocated; the assembler is unable to process these references in usual way.
- The assembler has no idea where any other control section will be located at execution time. Such references between control sections are called **external references.**
- The assembler generated information for each external reference that will allow the loader to perform the required linking.

Assembler uses two assembler directives namely

1. **EXTDEF (EXTERNAL DEFINITION)**
   - The EXTDEF statement in a control section names symbols called external symbols that are defined in this control section and may be used by other sections.

2. **EXTREF (EXTERNAL REFERENCE)**
   - The EXTREF statement names symbols that are used in this control section and are defined elsewhere.
   - The assembler must also include the information about the external references in the object program that will cause the loader to calculate proper address of the operand and insert when they are required.
   - To include the information about the external references we need two record types in the object program.

The two new record types are

1. **Define record**
   - It gives information about external symbol that are defined in this control section that is symbol named by EXTDEF.
2. **Refer record**
   - It lists symbols that are used as external references by this control section but the symbols are defined in other control section, that is, symbol named by EXTREF.

FORMAT OF DERFINE RECORD:

Col 1	D
Col 2-7	name of external symbol defined in this control section
Col 8-13	relative address of symbol within this control section
Col 14-73	repeat information in col 2-13 for other external symbol

FORMAT OF REFER RECORD:

Col 1	R
Col 2-7	name of external symbol defined in this control section
Col 8-73	name of other external reference symbol

The new format of modification record is of the following

FORMAT OF MODIFICATION RECORD:

Col 1	M
Col 2-7	starting address of the field to be modified, relative to the beginning of the control section
Col 8-9	length of the field to be modified in half bytes
Col10	modification flag (+ or -)
Col 11-16	external symbol whose value is to be added to or subtracted from the indicated field

Example Object program:

H^COPY^000000^001033

D^BUFFER^000033^BUFEND^001033^LENGTH^00002D

R^RDREC^WRREC

T^000000610^172027^4B100000^032023^290000^32007^4B1000^3F2FFC^032016^0F2016

T^000010^00^010003^0F2000A^4B1000^3F2000

T^000030^03^454F46

M^000004^05+RDREC

M^000011^05+WRREC

M^000024^05+WRREC

E^000000

12. Explain Assembler design options? (OR) Explain in detail about one pass and multi-pass assembler. (11 MARKS) (MAY 2012, 2017, 2018) (NOV 2012, 2014, 2015, 2016, 2017)

The Assembler design options are

1. One –Pass Assemblers
2. Multi-Pass Assemblers

(i) ONE PASS ASSEMBLERS

The main problem in designing the assembler using single pass was to resolve forward references. We can avoid to some extent the forward references by:

- Eliminating forward reference to data items, by defining all the storage reservation statements at the beginning of the program rather at the end.
- Unfortunately, forward reference to labels on the instructions cannot be avoided (forward jumping).
- To provide some provision for handling forward references by prohibiting forward references to data items.

There are two types of one-pass assemblers:

1. One that produces object code directly in memory for immediate execution **(Load- and-go assemblers).**
2. The other type produces the usual kind of object code for later execution.

Load-and-Go Assembler

- Load-and-go assembler generates their object code in memory for immediate execution.
- No object program is written out, no loader is needed.
- It is useful in a system with frequent program development and testing.
- The efficiency of the assembly process is an important consideration.
- Programs are re-assembled nearly every time they are run; efficiency of the assembly process is an important consideration**.**

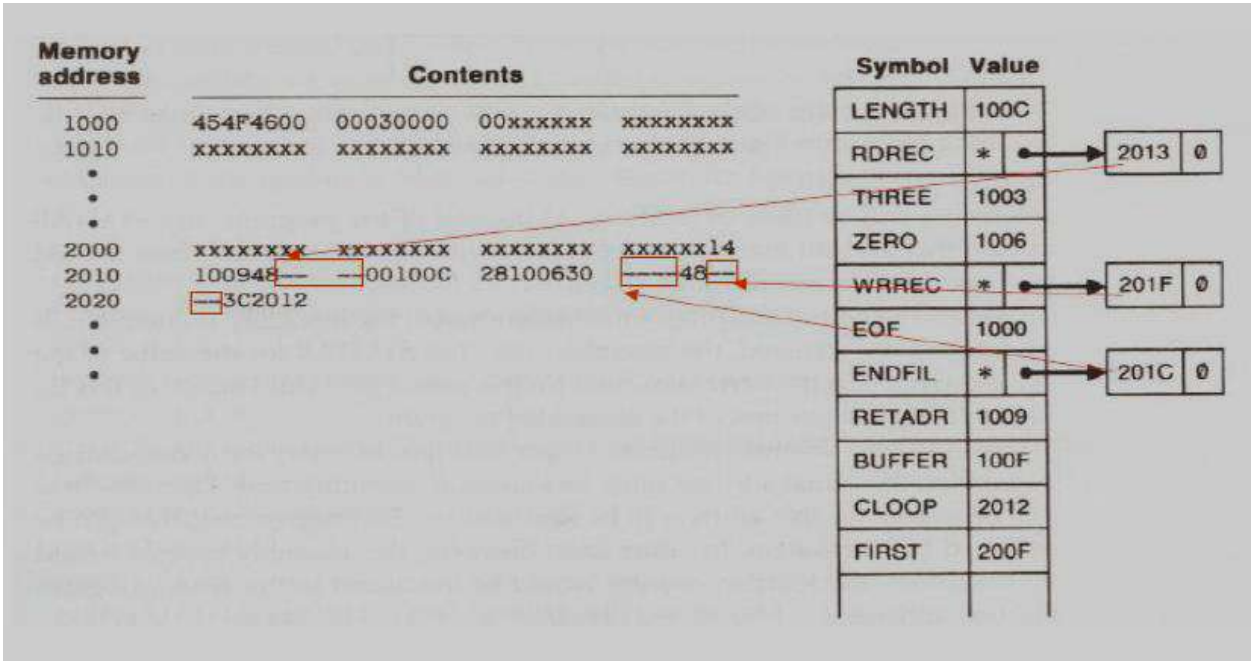
Forward Reference in One-Pass Assemblers

In load-and-Go assemblers when a forward reference is encountered:

- Omits the operand address if the symbol has not yet been defined.
- Enters this undefined symbol into SYMTAB and indicates that it is undefined.
- Adds the address of this operand address to a list of forward references associated with the SYMTAB entry.
- When the definition for the symbol is encountered, scans the reference list and inserts the address.
- At the end of the program, reports the error if there are still SYMTAB entries indicated undefined symbols.
- For Load-and-Go assembler
  - Search SYMTAB for the symbol named in the END statement and jumps to this location to begin execution if there is no error.

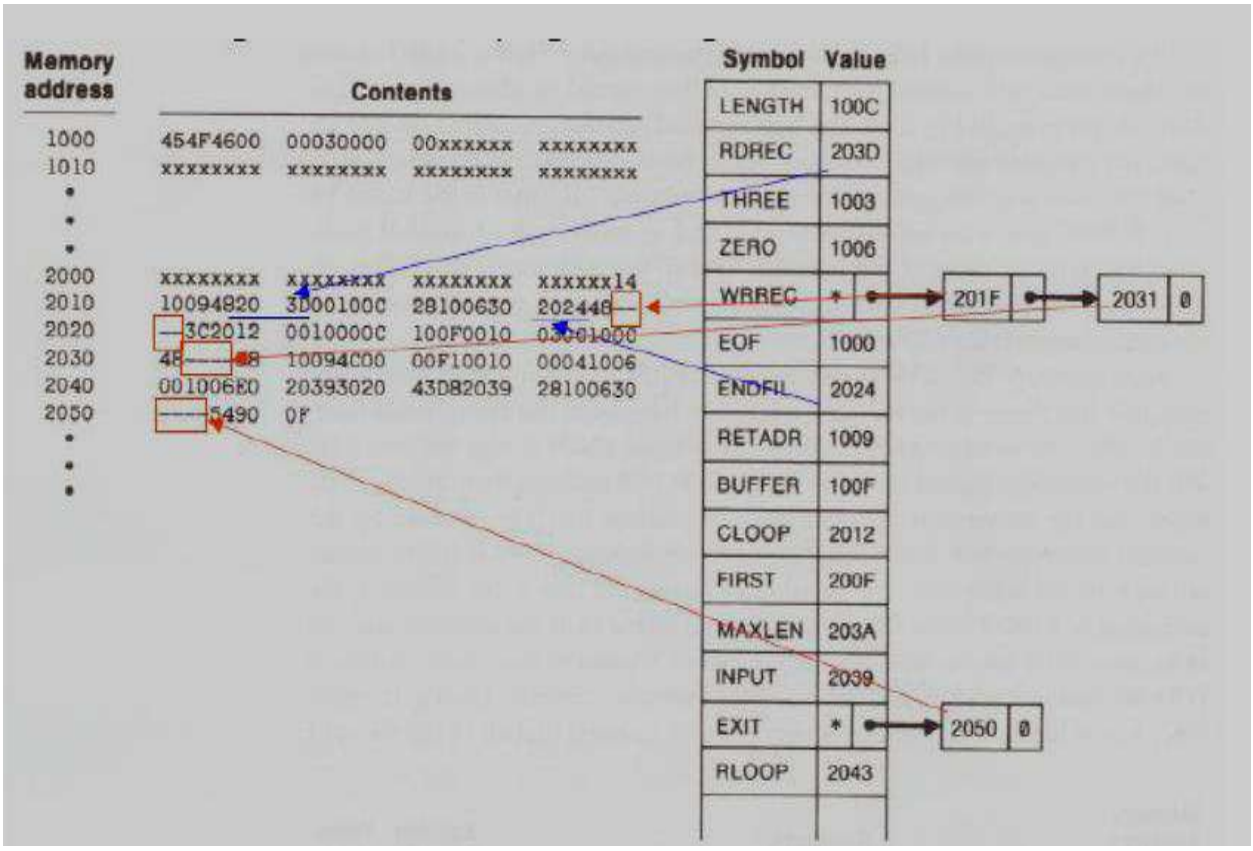
Example

Line	Loc	Source statement			Object code
0	1000	COPY	START	1000	
1	1000	EOF	BYTE	C'EOF'	454F46
2	1003	THREE	WORD	3	000003
3	1006	ZERO	WORD	0	000000
4	1009	RETADR	RESW	1	
5	100C	LENGTH	RESW	1	
6	100F	BUFFER	RESB	4096	
9		.			
10	200F	FIRST	STL	RETADR	141009
15	2012	CLOOP	JSUB	RDREC	48203D
20	2015		LDA	LENGTH	00100C
25	2018		COMP	ZERO	281006
30	201B		JEQ	ENDFIL	302024
35	201E		JSUB	WRREC	482062
40	2021		J	CLOOP	302012
45	2024	ENDFIL	LDA	EOF	001000
50	2027		STA	BUFFER	0C100F
55	202A		LDA	THREE	001003
60	202D		STA	LENGTH	0C100C
65	2030		JSUB	WRREC	482062
70	2033		LDL	RETADR	081009
75	2036		RSUB		4C0000
110		.			
115		.	SUBROUTINE TO READ RECORD INTO BUFFER		
120		.			
121	2039	INPUT	BYTE	X'F1'	F1
122	203A	MAXLEN	WORD	4096	001000
124		.			
125	203D	RDREC	LDX	ZERO	041006
130	2040		LDA	ZERO	001006
135	2043	RLOOP	TD	INPUT	E02039
140	2046		JEQ	RLOOP	302043
145	2049		RD	INPUT	D82039
150	204C		COMP	ZERO	281006
155	204F		JEQ	EXIT	30205B
160	2052		STCH	BUFFER,X	54900F
165	2055		TIX	MAXLEN	2C203A
170	2058		JLT	RLOOP	382043
175	205B	EXIT	STX	LENGTH	10100C
180	205E		RSUB		4C0000
195		.			
200		.	SUBROUTINE TO WRITE RECORD FROM BUFFER		
205		.			
206	2061	OUTPUT	BYTE	X'05'	05
207		.			
210	2062	WRREC	LDX	ZERO	041006
215	2065	WLOOP	TD	OUTPUT	E02061
220	2068		JEQ	WLOOP	302065
225	206B		LDCH	BUFFER,X	50900F
230	206E		WD	OUTPUT	DC2061
235	2071		TIX	LENGTH	2C100C
240	2074		JLT	WLOOP	382065
245	2077		RSUB		4C0000
255			END	FIRST	



One-Pass needs to generate object code

- If the operand contains an undefined symbol, use 0 as the address and write the Text record to the object program.
- Forward references are entered into lists as in the load-and-go assembler.
- When the definition of a symbol is encountered, the assembler generates another Text record with the correct operand address of each entry in the reference list.
- When loaded, the incorrect address 0 will be updated by the latter Text record containing the symbol definition.



Object Code Generated by One-Pass Assembler

```
HCOPY  001000001 07A
T001000094 54F46000003000000
T00200F151410094800000100C2810063000004800003C2012
T00201C022024
T0020241900100000C100F0010030C100C480000081009 4C0000F1001000
T00201302203D
T00203D1E041006001006E02039302043D82039281006300000549 00F2C203A382043
T002050022 05B
T00205B0710100C4C000005
T00201F022062
T002031022062
T0020621804 1006E02061302065 50900FDC20612C100C3820654C0000
E00200F
```

(ii) MULTI-PASS ASSEMBLERS

- EQU assembler directive is required to define symbol used on the right hand side be defined in the source program.

For a two pass assembler, forward references in symbol definition are not allowed:

ALPHA	**EQU**	**BETA**
BETA	**EQU**	**DELTA**
DELTA	**RESW**	**1**

- Symbol definition must be completed in pass 1.

Prohibiting forward references in symbol definition is not a serious inconvenience.

- Forward references tend to create difficulty for a person reading the program.

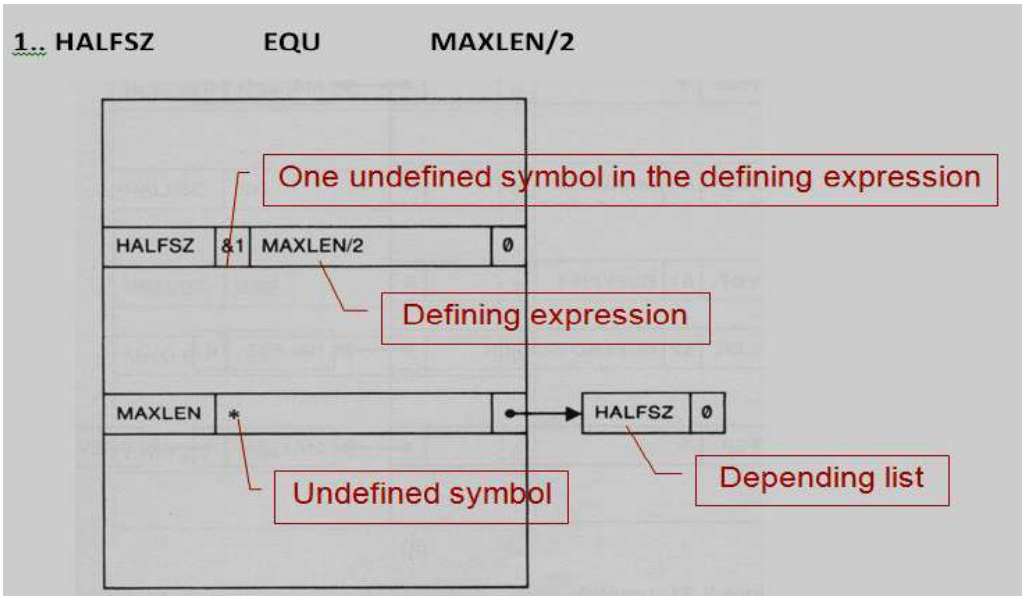
Implementation Issues for Modified Two-Pass Assembler

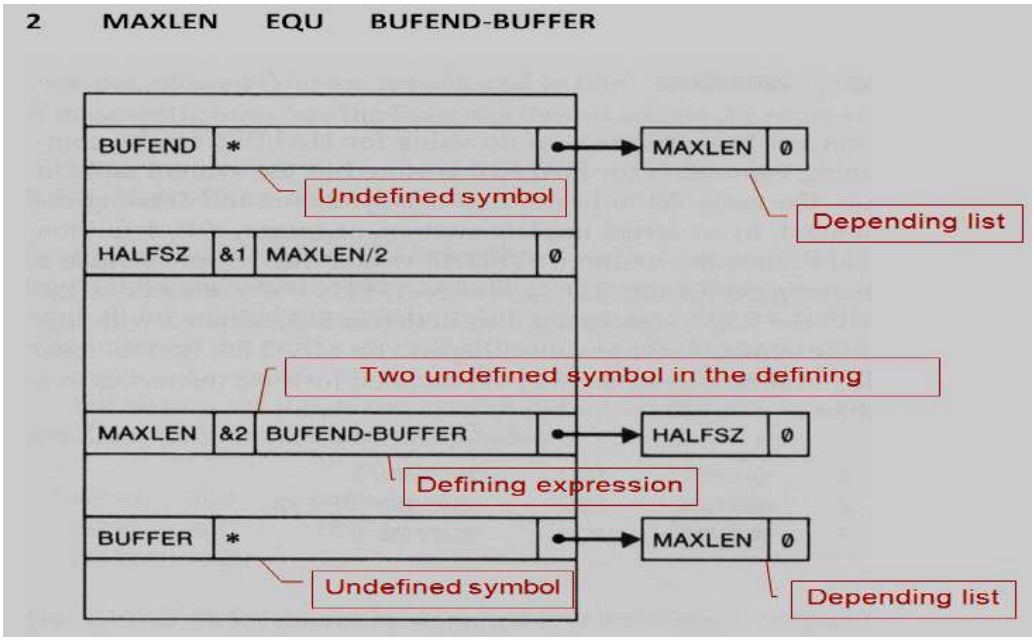
Implementation Issues when forward referencing is encountered in Symbol Defining statements:

- For a forward reference in symbol definition, we store in the SYMTAB:
  - The symbol name
  - The defining expression
  - The number of undefined symbols in the defining expression
- The undefined symbol (marked with a flag *) associated with a list of symbols depend on this undefined symbol.
- When a symbol is defined, we can recursively evaluate the symbol expressions depending on the newly defined symbol.

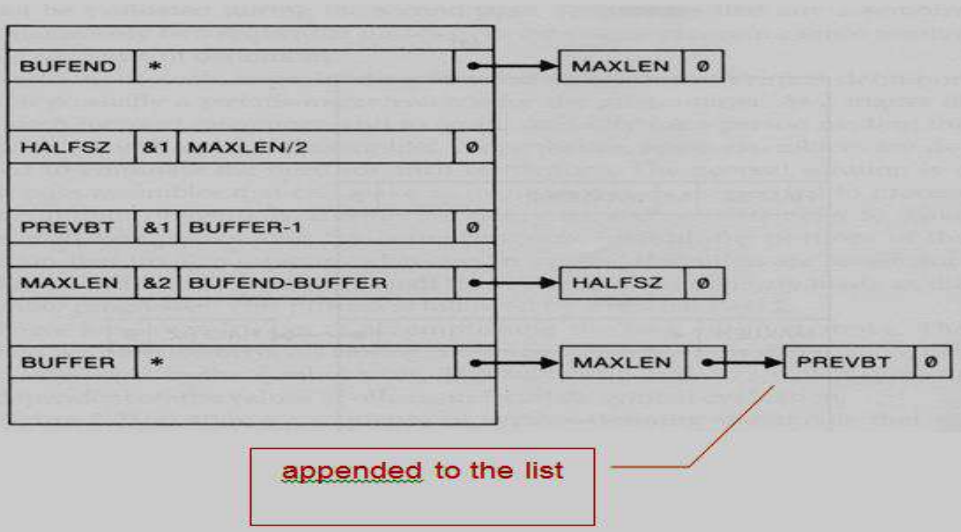
Multi-Pass Assembler Example Program

```
1       HALFSZ      EQU     MAXLEN/2
2       MAXLEN      EQU     BUFEND-BUFFER
3       PREVBT      EQU     BUFFER-1
                            .
                            .
                            .
4       BUFFER      RESB    4096
5       BUFEND      EQU     *
```

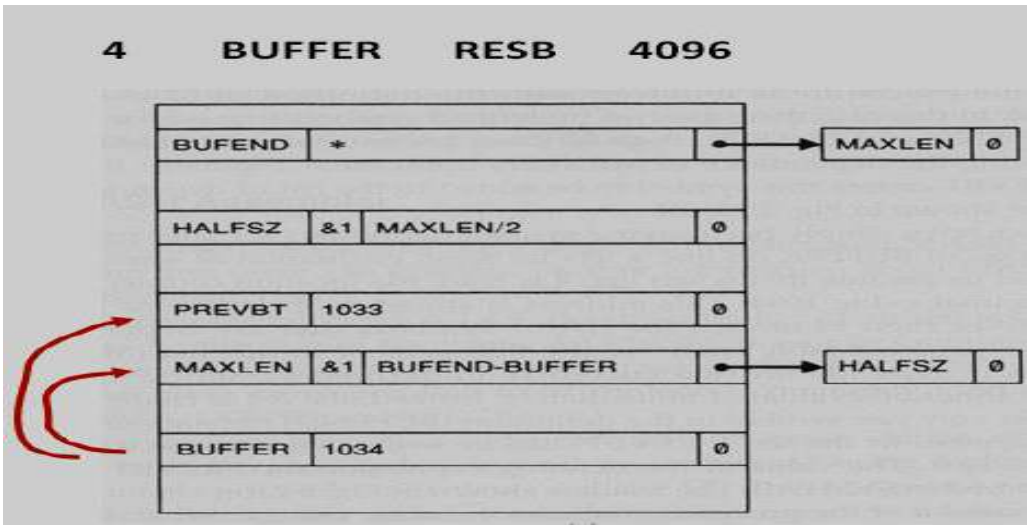




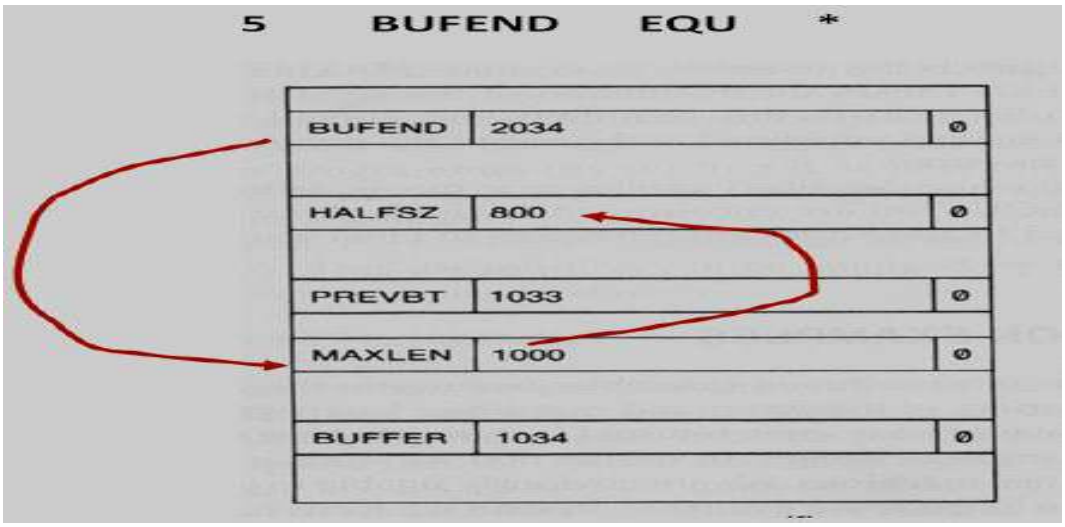
3 PREVBT EQU BUFFER-1



4 BUFFER RESB 4096



5 BUFEND EQU *



13. Write a program for SIC/XE to add 2 arrays containing 100 integers. (4 MARKS) (NOV 2017)

```
        LDS     #3          INITIALIZE REGISTER S TO 3
        LDT     #300        INITIALIZE REGISTER T TO 300
        LDX     #0          INITIALIZE INDEX REGISTER TO 0
ADDLP   LDA     ALPHA,X     LOAD WORD FROM ALPHA INTO REGISTER A
        ADD     BETA,X      ADD WORD FROM BETA
        STA     GAMMA,X     STORE THE RESULT IN A WORD IN GAMMA
        ADDR    S,X         ADD 3 TO INDEX VALUE
        COMPR   X,T         COMPARE NEW INDEX VALUE TO 300
        JLT     ADDLP       LOOP IF INDEX VALUE IS LESS THAN 300
        .
        .
        .
.                           ARRAY VARIABLES--100 WORDS EACH
ALPHA   RESW    100
BETA    RESW    100
GAMMA   RESW    100
```

PONDICHERRY UNIVERSITY QUESTIONS

2 MARKS

1. What is interpreter? **(NOV 2011, 2012) (MAY 2018) (Ref.Qn.No.10, Pg.no.6)**
2. What is Language Translator? **(NOV 2014) (Ref.Qn.No.1, Pg.no.5)**
3. What are the fields available in an assembly language instruction? **(NOV 2011) (Ref.Qn.No.29, Pg.no.11)**
4. Define Assembler? **(MAY 2012) (Ref.Qn.No.9, Pg.no.6)**
5. Define System Software. ? **(MAY 2012, 2015) (NOV 2012, 2016) (Ref.Qn.No.4, Pg.no.5)**
6. What is SIC machine? **(NOV 2014) (Ref.Qn.No.13, Pg.no.7)**
7. What is mean by OPTAB, SYMTAB? **(MAY 2012) (Ref.Qn.No.33,34, Pg.no.11,12)**
8. List the assembler directives. **(NOV 2012) (Ref.Qn.No.30, Pg.no.11)**
9. Define Instruction set. **(MAY 2013) (Ref.Qn.No.18, Pg.no.8)**
10. What is meant by Literal pool? **(MAY 2014) (Ref.Qn.No.41, Pg.no.13)**
11. What are literals? **(NOV 2015) (Ref.Qn.No.40, Pg.no.13)**
12. Define the basic functions of assembler. **(NOV 2014) (MAY 2016) (Ref.Qn.No.31, Pg.no.11)**
13. What is an External reference? **(NOV 2014) (Ref.Qn.No.48, Pg.no.14)**
14. How could literals be implemented in one pass assembler? **(MAY 2013) (Ref.Qn.No.12, Pg.no.7)**
15. State the difference between assembler and Interpreter. **(NOV 2013) (Ref.Qn.No.11, Pg.no.7)**
16. List the features of machine dependent Assembler. **(NOV 2013) (Ref.Qn.No.37, Pg.no.12)**
17. State the limitations of one pass assembler. **(MAY 2014) (Ref.Qn.No53., Pg.no.15)**
18. Compare Assembler and Compiler. **(NOV 2015) (Ref.Qn.No55., Pg.no.15)**
19. What is meant by machine independent assembler features? **(MAY 2016) (Ref.Qn.No.56, Pg.no.16)**
20. What is the difference between the instructions LDA #3 and LDA THREE? **(MAY 2016) (Ref.Qn.No.57, Pg.no.16)**
21. List out the internal data structures used in assemblers? **(NOV 2016) (Ref.Qn.No.58, Pg.no.16)**
22. Compare and Contrast Application and System Software. **(MAY 2017) (Ref.Qn.No.7, Pg.no.6)**
23. List out the roles of Operating System in program execution. **(MAY 2017) (Ref.Qn.No.59, Pg.no.16)**
24. Differentiate direct and indirect addressing Modes. **(NOV 2017) (Ref.Qn.No.60, Pg.no.16)**
25. What is the purpose of program counter? **(NOV 2017) (Ref.Qn.No.61, Pg.no.16)**
26. Differentiate between assembler and compiler. **(MAY 2018) (Ref.Qn.No.62, Pg.no.17)**

11 MARKS

NOV 2011 (REGULAR)

1. (a) Draw the format of an instruction and explain. **(5)** **(Ref.Qn.No.5,6 Pg.no.26,29)**

   (b) Describe the machine structure. **(6)** **(Ref.Qn.No.5, Pg.no.26)**

   (OR)

2. Describe the features of machine-independent assembler. **(Ref.Qn.No.11, Pg.no.43)**

MAY 2012 (ARREAR)

1. List out and discuss the machine dependent assembler features. **(Ref.Qn.No.10, Pg.no.39)**

   (OR)

2. Briefly explain about assembler design options. **(Ref.Qn.No.12, Pg.no.51)**

NOV 2012 (REGULAR)

1. List out and discuss the machine independent assembler features. **(Ref.Qn.No.11, Pg.no.43)**

   (OR)

2. Briefly explain about one-pass assembler and multi-pass assembler. **(Ref.Qn.No.12, Pg.no.51)**

MAY 2013 (ARREAR)

1. What is the important machine structures used in the design of system software? Discuss. **(Ref.Qn.No.5, Pg.no.26)**

   (OR)

2. Give two example programs SIC operations. **(Ref.Qn.No.7, Pg.no.31)**

NOV 2013 (REGULAR)

1. Explain the simplified instructional computer (SIC) architecture in detail. **(Ref.Qn.No.5, Pg.no.26)**

   (OR)

2. Explain the basic assembler functions and the steps in the design of an assembler. **(Ref.Qn.No.8, Pg.no.32)**

MAY 2014 (ARREAR)

1. Explain in detail about Instruction formats, Instruction set and addressing modes of SIC. **(Ref.Qn.No.5, Pg.no.26)**

   (OR)

2. Explain briefly about machine Independent assembler features. **(Ref.Qn.No.11, Pg.no.43)**

NOV 2014 (REGULAR)

1. Explain the SIC machine architecture. **(Ref.Qn.No.5, Pg.no.26)**

   (OR)

2. Explain in detail about one pass and multi-pass assembler. **(Ref.Qn.No.12, Pg.no.51)**

MAY 2015 (ARREAR)

1. (a). Compute the target address for the SIC/XE instruction of format 3 stored at address 415 (hexadecimal) and whose machine code is "17AFF2 (hexadecimal)". Assume that the content of index register (X) is 12. **(3)**

   (b). Convert the following instructions in SIC into its equivalent machine code. Assume the opcode of ADD is 18 (hexadecimal) and buffer's value is 120H and the address of the ADD instruction is 11CH and contents of index register are 24H. **(8)**

   (i) ADD buffer

   (ii) ADD buffer, X

   (OR)

2. Explain the various processing done in Pass-I for a 2 pass assembler for SIC processor. **(Ref.Qn.No.8, Pg.no.35)**

NOV 2015 (REGULAR)

1. Write an algorithm for a multi-pass Assemblers. **(Ref.Qn.No.12, Pg.no.51)**

   (OR)

2. Explain the different addressing modes with suitable formats. **(Ref.Qn.No6., Pg.no.28)**

MAY 2016 (ARREAR)

1. Write the algorithm for Pass 1 and Pass 2 Assembler. **(Ref.Qn.No.8, Pg.no.35)**

   (OR)

2. (a) Give come applications of operating system. (5) **(Ref.Qn.No.4, Pg.no.23)**

   (b) Explain the symbol defining statements generally used in assemblers. (6) **(Ref.Qn.No.11, Pg.no.43)**

NOV 2016 (REGULAR)

1. Explain about SIC. **(Ref.Qn.No.5, Pg.no.26)**

   (OR)

2. Write about multi-pass Assemblers. **(Ref.Qn.No.12, Pg.no.51)**

MAY 2017 (ARREAR)

1. Elaborate in detail about various addressing modes and their use case. **(Ref.Qn.No.6, Pg.no.28)**

   (OR)

2. Write in detail about Multi-pass assembler. **(Ref.Qn.No.12, Pg.no.51)**

NOV 2017 (REGULAR)

1. (a) Explain instruction formats with reference to SIC/XE machine architecture. **(7)** **(Ref.Qn.No.6, Pg.no.28)**

   (b) Write a program for SIC/XE to add 2 arrays containing 100 integers. **(4)** **(Ref.Qn.No.13, Pg.no.57)**

   (OR)

2. (a). How are forward references handled by multi-pass assembler? Illustrate with an example. **(7)** **(Ref.Qn.No.12, Pg.no.54)**

   (b). What is the need for two-pass assembler? Reason out with simple example? **(4)** **(Ref.Qn.No.12, Pg.no.51)**

MAY 2018 (ARREAR)

1. (a). Explain the machine architecture. **(6)** **(Ref.Qn.No.5, Pg.no.26)**

   (b). Discuss about the addressing modes used in the machine. **(6)** **(Ref.Qn.No.6, Pg.no.29)**

   (OR)

2. Design a Two pass assembler. Explain the tables constructed by it for an example. **(Ref.Qn.No.12, Pg.no.51)**